23CSE455 Design Patterns

Case Study Report

Team: A12

Case study: OTT / video streaming application using React / Next.js — *Design Pattern Analysis in a Next.js-Based Video Streaming Application*

Application: netflx-web (Netflix clone, Next.js 13 app router, tRPC, NextAuth, Prisma, Stripe) by sadmann7. Faculty approval: approved by the course faculty for use in this case study.

Date: 2026-10-30 (first analysis 2026-10-10, exhaustive second pass 2026-10-29/30)

Submission repository: 23CSE455_DP_CaseStudy, folder `A12/`

Students

Roll number	Student name	Batch
AM.SC.U4CSE23161	Swaminathan A	CSE B
AM.SC.U4CSE23207	Anna Ouseph	CSE C
AM.SC.U4CSE23230	Leya Paul	CSE C
AM.SC.U4CSE23242	Pooja Sathyan	CSE C
AM.SC.U4CSE23069	Rahul Rajan	CSE A

1 Application and architecture

Original source: `https://github.com/sadmann7/netflx-web`, branch `main`, no release tags; baseline commit `676e5ddd15d68fe6b83c2f9cc2494c6ece8b2258` (2024-06-12, "Merge pull request #11"); accessed 2026-10-10. **Licence: none** — the repository has no LICENSE file, so all rights are reserved by the author; the code is used only for this academic analysis. Snapshot: `data/code/`.

Environment: TypeScript 5.0.4 on Node.js; Next.js 13.4.4 (app router + pages router for API routes), React 18.2.0, tRPC 10.28.0, NextAuth 4.22.1 with @next-auth/prisma-adapter 1.0.6 (Google OAuth), Prisma 4.14.1 on MySQL (`relationMode = "prisma"`), Stripe 12.7.0, Zustand 4.3.8, @tanstack/react-query 4.29.7, Tailwind CSS 3.3.2; catalogue data from the TMDB REST API. Analysis tools (2026-10-10, Linux sandbox): Node.js 22.22.0, yarn 1.22.22, vitest 1.6.0, lizard 1.24.1, PlantUML + Graphviz. Second pass and Part 3 re-run (2026-10-29/30, Windows 11 x64): Temurin JDK 21.0.12, Python 3.12.10, Node.js 26.13.0, g++ 16.2.0 (WinLibs UCRT), lizard 1.24.3, PlantUML 1.2024.7 (built-in smetana layout); dependency sources inspected with `npm pack` at the versions of `yarn.lock`.

Scope: Inspected: `src/server/**`, `src/lib/**`, `src/config/**`, `src/stores/**`, `src/app/**`, `src/components/**`, `src/@trpc/next-layout/**`, `src/pages/**`, `src/hooks/**`, `src/context/**`, `src/env.mjs`, `prisma/schema.prisma` — in the second pass every one of these files was read completely — and dependency code where a role is implemented (next-auth, prisma-adapter, zustand, tRPC server/client/react-query/next, react-query, next). Excluded: `components/ui/*` (generated shadcn UI primitives), `styles/`, `public/`, `app/(docs)` (static help/terms text) and `app/api/og` (Open Graph image) — none of them belongs to the seven case-study areas. The area-to-file map is in `data/test_result/part1/candidates.md`.

Architecture: Figure 1 (source `Part1/pattern_architecture/00_architecture.puml`).

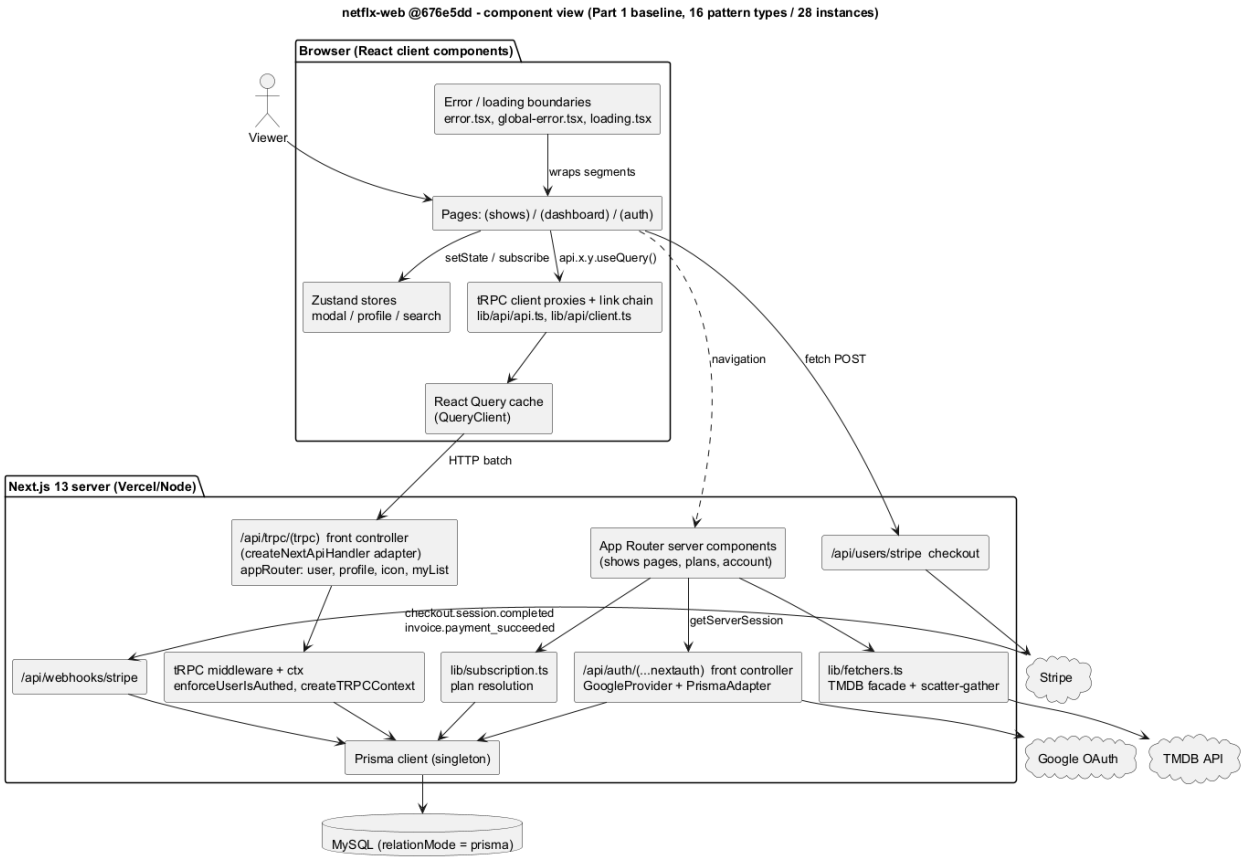


Figure 1: Component view of netflx-web at the baseline.

Components. *Browser*: React client components (shows carousels, modal, profile screens, forms) keep UI state in three Zustand stores (modal, profile, search) and call the server through two tRPC client proxies backed by one React-Query cache. *Server*: App-router server components render pages and fetch catalogue rows from TMDB through `lib/fetchers.ts`; the tRPC API (`/api/trpc`, pages router) serves user, profile, icon and My-List procedures behind an authentication middleware; NextAuth (`/api/auth`) signs users in with Google and stores users/sessions through the Prisma adapter; Stripe checkout (`/api/users/stripe`) and a Stripe webhook (`/api/webhooks/stripe`) maintain the subscription fields on the User row. Every server module reads configuration through the `env` proxy. Two *front controllers* receive all API traffic: `/api/trpc/[trpc]` (all 19 procedures, adapted to Next.js by `createNextApiHandler`) and `/api/auth/[...nextauth]` (all auth actions). Route segments are wrapped by Next.js error boundaries (`error.tsx`, `global-error.tsx`) and a Suspense loading boundary (`(shows)/loading.tsx`).

Request flow (adding a title to My List). A viewer clicks a poster → `ShowCard` publishes `{show, open}` to `useModalStore` → `ShowsContainer` renders `ShowModal`, which fetches the trailer from TMDB → "Add to My List" calls `api.myList.create.useMutation` on the tRPC proxy → HTTP batch to `/api/trpc` → `createTRPCContext` loads the session → `enforceUserIsAuthed` passes the request on → the resolver loads the profile with its list through the Prisma singleton and inserts a `MyShow` row → the modal invalidates the cached `myList.getAll` query, so the next read reloads it. **Run instructions:** `README.md` (team) and `data/code/README.md` (original). Setup: copy `.env.example` to `.env` (MySQL URL, Google OAuth, TMDB key, Stripe keys), `yarn install` (runs `prisma generate`), `npx prisma db push`, `yarn dev`, `yarn stripe:listen`. **Actual result in the analysis environment:** `npx tsc -noEmit` passes on the baseline (`data/test_result/baseline/typecheck.log`); `next build` stops at `next/font` because the sandbox cannot download Inter from Google Fonts (`data/test_result/baseline/build.log`) — an environment limit, not a code error; the baseline repository contains no tests. Prisma engines could not be downloaded

either, so the database was replaced by an in-memory fake in all tests.

2 Part 1 Pattern discovery

Method: First pass (2026-10-10). (1) Listed the entry files of each of the seven areas and read them completely. (2) Followed every import/call one hop, including into `node_modules` when a role is played by a dependency (versions pinned by `yarn.lock`). (3) For each candidate, recorded intent, the role relationships and the delegation that happens at run time; a candidate was confirmed only when all three are visible in code, never from a name, comment or LLM statement alone. (4) Line ranges were checked by script: `data/test_result/part1/verify_locations.py` (all cited ranges exist and start at the named symbol) and an independent bracket-matching locator (`data/test_result/part2/locate.py`) for end lines, which found and fixed two wrong end lines. (5) Rejected and uncertain candidates were recorded with reasons. The first pass was done with an LLM (section 5) and every claim was then checked against the source; it found 10 types / 15 instances. **Second, exhaustive pass (2026-10-29/30).** (6) Every in-scope file was read top to bottom, not only the area entry files, and checked against the GoF catalogue, architectural (PoEAA, Core J2EE), concurrency/integration and React/Next.js idioms. (7) Dependency roles were verified in the exact package versions of `yarn.lock` (`npm pack` of @trpc/server, client, react-query, next 10.28.0, next-auth 4.22.1, next 13.4.4) with file:line references. (8) The location check was re-run over all **156** cited ranges: 0 problems (`data/test_result/part1/location_check.log`). The second pass added 6 types and 13 instances and corrected one role of an existing record (Transport of the Pages-Router tRPC Client Proxy, see its record).

2.1 Totals at the baseline

Confirmed pattern types: Chain of Responsibility, Singleton, Observer, Decorator, Adapter, Strategy, Facade, Composite, Proxy, Read-Through Cache, Front Controller, Builder, Context Object, Error Boundary, Suspense Boundary, Scatter-Gather — **16**

Confirmed pattern instances: 28 independent collaborations (table below).

Concrete participant classes: 50 implementations of variable roles (ConcreteHandler, ConcreteObserver, ConcreteStrategy, ConcreteComponent/Decorator, Adapter, Singleton, Facade module, Composite routers, Proxy, Cache, FrontController, ConcreteBuilder, ContextFactory, boundary fallbacks, scatter/gather functions). Many are React function components or modules, not classes; we do not invent classes for them.

Unique participant symbols: 85 distinct mapped symbols in the repository (e.g. `ShowsContainer`, `prisma`, `AppRouter` counted once although they take part in several instances) plus **29** dependency symbols (e.g. `PrismaAdapter`, zustand `persist`); list in `data/test_result/part1/part1_patterns.py`.

Rejected and uncertain candidates: 11 rejected, **5 uncertain** (table at the end of this section, full list with locations in `data/test_result/part1/candidates.md`). Observation scope: static reading of the baseline source and of the pinned dependency sources, plus type checking; no run with real Stripe, TMDB or Google credentials.

2.2 Pattern inventory

Pattern type	Instance names	Instances	Concrete
Chain of Responsibility	Authenticated Procedure Pipeline; tRPC Client Link Chain	2	3
Singleton	Shared Prisma Database Client; Stripe API Client	2	2
Observer	Show Modal Selection; Active Profile Selection; Search Results Broadcast; Search Input DOM Event Listeners	4	17
Decorator	Persisted Profile Store	1	3
Adapter	NextAuth Prisma Persistence Adapter; tRPC Next.js API Handler Adapter	2	2
Strategy	Sign-in Provider Strategy; Wire Serialization Transformer	2	2
Facade	TMDB Catalogue Facade	1	1
Composite	API Router Tree	1	5
Proxy	Pages-Router tRPC Client Proxy; App-Router tRPC Client Proxy; Environment Secret Guard	3	3
Read-Through Cache	My List Query Cache; Profile and Account Query Cache	2	2
Front Controller	tRPC API Front Controller; NextAuth Auth Route Front Controller	2	2
Builder	tRPC Procedure Builder	1	2
Context Object	tRPC Request Context	1	1
Error Boundary	Shows Segment Error Boundary; Root Global Error Boundary	2	2
Suspense Boundary	Shows Loading Placeholder	1	1
Scatter-Gather	TMDB Row Scatter-Gather	1	2
Total		28	50

Mapping to the seven areas. User management: Active Profile Selection, Persisted Profile Store, API Router Tree, Profile and Account Query Cache. Video catalogue: TMDB Catalogue Facade, TMDB Row Scatter-Gather, Show Modal Selection, Search Results Broadcast, Search Input DOM Event Listeners, Shows Loading Placeholder, Shows Segment Error Boundary. Subscription: Stripe API Client (plans resolved in `lib/subscription.ts`). Authentication: Authenticated Procedure Pipeline, Sign-in Provider Strategy, NextAuth Prisma Persistence Adapter, NextAuth Auth Route Front Controller, Shared Prisma Database Client. Content access: Authenticated Procedure Pipeline and tRPC Request Context (server), Environment Secret Guard (configuration secrets); the profile PIN gate is only an uncertain candidate. Caching: My List Query Cache, Profile and Account Query Cache, (rejected) server caching proxy. Cross-cutting infrastructure used by all areas: tRPC API Front Controller, tRPC Next.js API Handler Adapter, tRPC Procedure Builder, tRPC Client Link Chain, Wire Serialization Transformer, Root Global Error Boundary. Recommendation: **no pattern** — the home page shows fixed TMDB lists (trending, top rated, five genre ids); no user signal is used.

2.3 One record for each pattern instance

Authenticated Procedure Pipeline

Pattern type and category: Chain of Responsibility — Behavioral (GoF); tRPC middleware idiom

Purpose and origin: Problem: every private procedure must reject anonymous callers. Origin: application (create-t3-app scaffold) on framework.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Handler	tRPC MiddlewareFunction created by t.middleware	dependency @trpc/server@10.28.0
ConcreteHandler	enforceUserIsAuthed	`data/code/src/server/api/trpc.ts:109-119`
Chain assembly	protectedProcedure = t.procedure.use(enforceUserIsAuthed)	`data/code/src/server/api/trpc.ts:129`
Terminal handler	procedure resolvers e.g. profileRouter.getAll	`data/code/src/server/api/routers/profile.ts:7-15`
Client	createNextApiHandler (tRPC HTTP entry point)	`data/code/src/pages/api/trpc/[trpc].ts:8-19`

Evidence: Problem: every private procedure must reject anonymous callers. enforceUserIsAuthed inspects ctx.session and either throws TRPCError UNAUTHORIZED (stops the chain) or calls next({ctx}) with a narrowed session, passing the request to the next link. protectedProcedure attaches the handler once; 18 of 19 procedures in user/profile/icon/myList routers are built on it, so the check runs before each resolver without being repeated.

Diagram: `Part1/pattern_architecture/01_cor_authenticated_procedure_pipeline.puml` (editable), `Part1/pattern_architecture/01_cor_authenticated_procedure_pipeline.png`

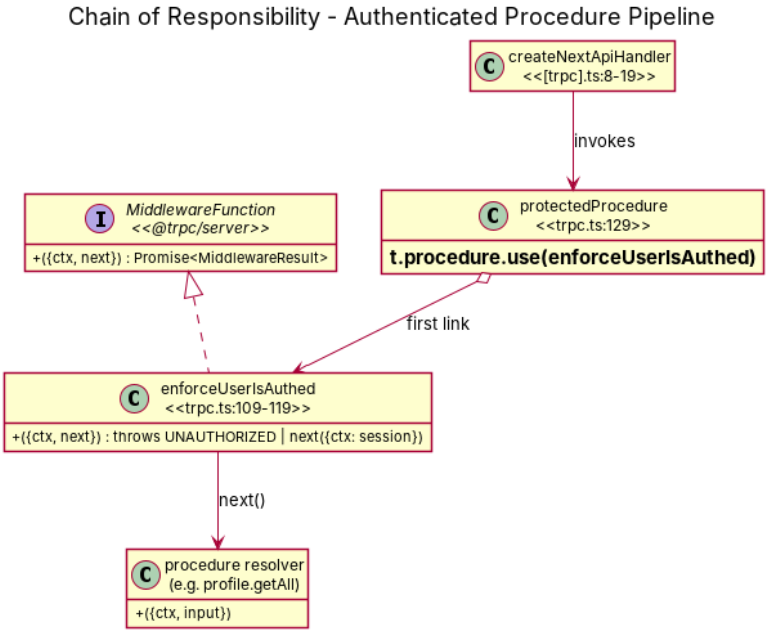


Figure 2: Authenticated Procedure Pipeline

Verification: Confirmed. Characterization tests: anonymous caller -> UNAUTHORIZED, signed-

in caller reaches the resolver with its id, public procedure stays open (`tests/characterization.test.ts`). Alternative considered: Decorator (procedure.use wraps) — rejected because each middleware decides whether to pass the request on, which is the defining CoR property. Reviewer: **[TEAM TO FILL: roll no./name]**

Shared Prisma Database Client

Pattern type and category: Singleton — Creational (GoF)

Purpose and origin: Problem: Next.js hot reload re-evaluates modules, which would open a new connection pool each time. Origin: application (create-t3-app scaffold).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Singleton	prisma (PrismaClient instance)	`data/code/src/server/db.ts:9-14`
Instance holder	globalForPrisma (globalThis cache)	`data/code/src/server/db.ts:5-7,16`
Client	createInnerTRPCContext	`data/code/src/server/api/trpc.ts:47-52`
Client	authOptions.adapter = PrismaAdapter(prisma)	`data/code/src/server/auth.ts:48`
Client	getUserSubscriptionPlan	`data/code/src/lib/subscription.ts:6-53`
Client	Stripe webhook POST	`data/code/src/app/api/webhooks/stripe/route.ts:8-76`

Evidence: Problem: Next.js hot reload re-evaluates modules, which would open a new connection pool each time. db.ts reuses globalForPrisma.prisma if present (??) and only constructs PrismaClient once, then stores it on globalThis outside production. All server code imports the same ‘prisma‘ export.

Diagram: `Part1/pattern_architecture/02_singleton_shared_prisma_client.puml` (editable), `Part1/pattern_architecture/02_singleton_shared_prisma_client.png`

Verification: Confirmed by reading; behaviour reproduced in Part 3 problem 02 (9/9 cases incl. hot reload). Alternative: plain module export — the explicit globalThis guard makes it a deliberate singleton. Reviewer: **[TEAM TO FILL: roll no./name]**

Stripe API Client

Pattern type and category: Singleton — Creational (GoF); ES-module singleton idiom

Purpose and origin: One configured Stripe client (API key, apiVersion 2022-11-15) is created at module load and exported; the ES module cache guarantees a single instance per server process. Origin: application.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Singleton	stripe (Stripe instance)	`data/code/src/lib/stripe.ts:4-7`
Client	Stripe webhook POST	`data/code/src/app/api/webhooks/stripe/route.ts:8-76`
Client	checkout/portal POST	`data/code/src/app/api/users/stripe/route.ts:14-115`
Client	PlansPage	`data/code/src/app/(auth)/login/plans/page.tsx:17-60`
Client	AccountPage	`data/code/src/app/(dashboard)/account/page.tsx:16-61`

Evidence: One configured Stripe client (API key, apiVersion 2022-11-15) is created at module load and exported; the ES module cache guarantees a single instance per server process. Four call sites share it instead of constructing their own. No global guard as in db.ts, so it relies on module caching only (noted as weaker form).

Diagram: `Part1/pattern_architecture/03_singleton_stripe_client.puml` (editable), `Part1/pattern_architecture/03_singleton_stripe_client.png`

Verification: Confirmed as module-level singleton; weaker form (no guard, relies on the ES module cache). Reviewer: **[TEAM TO FILL: roll no./name]**

Show Modal Selection

Pattern type and category: Observer — Behavioral (GoF); Zustand store subscription

Purpose and origin: Selecting a poster calls useModalStore.setState({show, open, play}) (shows-carousel.tsx:105-109). Origin: application on dependency (zustand@4.3.8).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Subject	zustand StoreApi (subscribe/setState) created by create()	dependency zustand@4.3.8
ConcreteSubject	useModalStore	`data/code/src/stores/modal.ts:13-20`
Publisher (calls setState)	ShowCard onClick	`data/code/src/components/shows-carousel.tsx:93-113`
ConcreteObserver	ShowsContainer	`data/code/src/components/shows-container.tsx:22-65`
ConcreteObserver	ShowModal	`data/code/src/components/show-modal.tsx:30-296`
ConcreteObserver	Hero	`data/code/src/components/hero.tsx:17-102`
ConcreteObserver	ShowsGrid	`data/code/src/components/shows-grid.tsx:13-48`

Evidence: Selecting a poster calls useModalStore.setState({show, open, play}) (shows-carousel.tsx:105-109). Components that called useModalStore() are subscribed by the hook and re-render when state changes: ShowsContainer/ShowsGrid open ShowModal, ShowModal fetches the trailer for the new show (show-modal.tsx:45-74). Publisher and observers never reference each other.

Diagram: `Part1/pattern_architecture/04_observer_show_modal_selection.puml` (editable), `Part1/pattern_architecture/04_observer_show_modal_selection.png`

Verification: Confirmed by reading the store and its subscribers; publisher and observers never reference each other. Reviewer: **[TEAM TO FILL: roll no./name]**

Active Profile Selection

Pattern type and category: Observer — Behavioral (GoF); Zustand store subscription

Purpose and origin: The selected profile and the PIN-gate flag live in one store. Origin: application on dependency (zustand@4.3.8).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Subject	zustand StoreApi	dependency zustand@4.3.8
ConcreteSubject	useProfileStore	`data/code/src/stores/profile.ts:12-27`
Publisher	ProfilesScreen profile button / SiteHeader menu / siteConfig sign-out	`data/code/src/components/screens/profiles-screen.tsx:86-91`; `data/code/src/components/layouts/site-header.tsx:150-155`; `data/code/src/config/site.ts:57`
ConcreteObserver	ProfilesScreen	`data/code/src/components/screens/profiles-screen.tsx:24-144`
ConcreteObserver	PinForm	`data/code/src/components/forms/pin-form.tsx:10-104`
ConcreteObserver	ShowsContainer	`data/code/src/components/shows-container.tsx:22-65`
ConcreteObserver	ShowModal	`data/code/src/components/show-modal.tsx:30-296`
ConcreteObserver	SiteHeader	`data/code/src/components/layouts/site-header.tsx:35-261`
ConcreteObserver	MyShows	`data/code/src/components/my-shows.tsx:16-54`

Evidence: The selected profile and the PIN-gate flag live in one store. Setting {profile, pinForm} makes ProfilesScreen switch between profile picker, PinForm and the app shell (profiles-screen.tsx:43-143); ShowsContainer/ShowModal/MyShows re-query My List for the new profile id. Six components subscribe; three places publish.

Diagram: `Part1/pattern_architecture/05_observer_active_profile_selection.puml` (editable), `Part1/pattern_architecture/05_observer_active_profile_selection.png`

Verification: Confirmed; six subscribing components, three publishers. Reviewer: **[TEAM TO FILL: roll no./name]**

Search Results Broadcast

Pattern type and category: Observer — Behavioral (GoF); Zustand store subscription

Purpose and origin: SiteHeader writes query and results into useSearchStore; when query is non-empty ShowsContainer swaps carousels for ShowsGrid (shows-container.tsx:40-42) and Hero hides itself (hero.tsx:29-31); MainNav clears the query on navigation (main-nav.tsx:36-37). Origin: application on dependency (zustand@4.3.8).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Subject	zustand StoreApi	dependency zustand@4.3.8
ConcreteSubject	useSearchStore	`data/code/src/stores/search.ts:11-16`
Publisher	SiteHeader.searchShowsByQuery	`data/code/src/components/layouts/site-header.tsx:51-55`
ConcreteObserver	SiteHeader	`data/code/src/components/layouts/site-header.tsx:35-261`
ConcreteObserver	ShowsContainer	`data/code/src/components/shows-container.tsx:22-65`
ConcreteObserver	Hero	`data/code/src/components/hero.tsx:17-102`
ConcreteObserver	MainNav	`data/code/src/components/layouts/main-nav.tsx:24-142`
ConcreteObserver	MyShows	`data/code/src/components/my-shows.tsx:16-54`

Evidence: SiteHeader writes query and results into useSearchStore; when query is non-empty ShowsContainer swaps carousels for ShowsGrid (shows-container.tsx:40-42) and Hero hides itself (hero.tsx:29-31); MainNav clears the query on navigation (main-nav.tsx:36-37). The search box does not know which views react.

Diagram: `Part1/pattern_architecture/06_observer_search_results_broadcast.puml` (editable), `Part1/pattern_architecture/06_observer_search_results_broadcast.png`

Verification: Confirmed; the search box does not know which views react. Reviewer: **[TEAM TO FILL: roll no./name]**

Persisted Profile Store

Pattern type and category: Decorator — Structural (GoF); Zustand middleware

Purpose and origin: persist and devtools each take a StateCreator and return a StateCreator with the same shape, wrapping set/get to add sessionStorage persistence and Redux-devtools reporting. Origin: application on dependency (zustand/middleware).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Component	StateCreator<ProfileState>	dependency zustand@4.3.8 (vanilla.d.ts)
ConcreteComponent	inline creator (set) => ({profile, setProfile, pinForm, setPinForm})	`data/code/src/stores/profile.ts:15-20`
ConcreteDecorator	persist(..., {name, storage: sessionStorage})	`data/code/src/stores/profile.ts:14-25`
ConcreteDecorator	devtools(...)	`data/code/src/stores/profile.ts:13-26`
Client	create<ProfileState>()	`data/code/src/stores/profile.ts:12`

Evidence: persist and devtools each take a StateCreator and return a StateCreator with the same shape, wrapping set/get to add sessionStorage persistence and Redux-devtools reporting. They are stacked devtools(persist(creator)); the modal and search stores use the undecorated creator, showing the behaviour is added per object without changing the creator.

Diagram: `Part1/pattern_architecture/07_decorator_persisted_profile_store.puml` (editable), `Part1/pattern_architecture/07_decorator_persisted_profile_store.png`

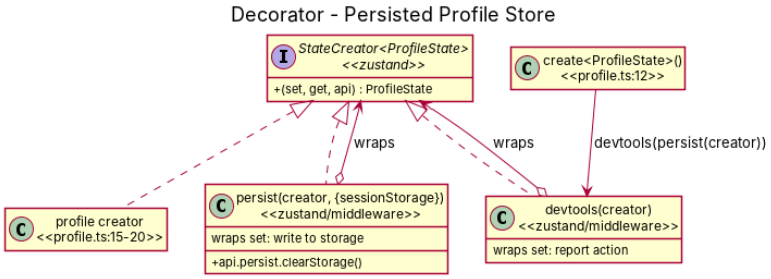


Figure 3: Persisted Profile Store

Verification: Confirmed; same StateCreator interface in and out; other stores use the undecorated creator. Reviewer: **[TEAM TO FILL: roll no./name]**

NextAuth Prisma Persistence Adapter

Pattern type and category: Adapter — Structural (GoF)

Purpose and origin: NextAuth persists users, accounts and sessions only through its Adapter interface (createUser, getUserByEmail, linkAccount, ...). Origin: dependency (@next-auth/prisma-adapter@1.0.6) configured by application.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Target	Adapter (next-auth adapter interface)	dependency next-auth@4.22.1 adapters.d.ts:57
Adapter	PrismaAdapter(p)	dependency @next-auth/prisma-adapter@1.0.6 dist/index.js:206-264
Adaptee	prisma (PrismaClient)	`data/code/src/server/db.ts:9-14`
Client	authOptions.adapter used by NextAuth handler	`data/code/src/server/auth.ts:38-64`; `data/code/src/pages/api/auth/[...nextauth].ts:4`

Evidence: NextAuth persists users, accounts and sessions only through its Adapter interface (createUser, getUserByEmail, linkAccount, ...). PrismaAdapter returns an object implementing those methods by translating them to Prisma calls, e.g. createUser: (data) => p.user.create({data}) and getUserByEmail: (email) => p.user.findUnique({where:{email}}) (index.js:208-210). The app wires it with adapter: PrismaAdapter(prisma).

Diagram: `Part1/pattern_architecture/08_adapter_nextauth_prisma.puml` (editable), `Part1/pattern_architecture/08_adapter_nextauth_prisma.png`

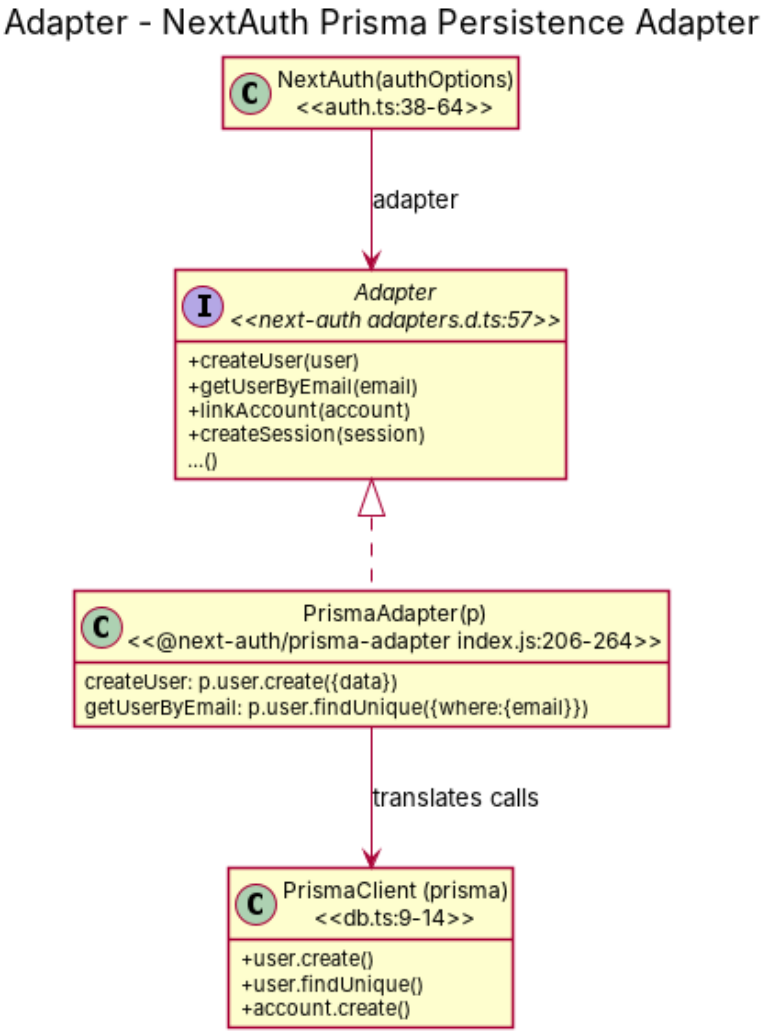


Figure 4: NextAuth Prisma Persistence Adapter

Verification: Confirmed by reading the adapter source in node_modules (createUser -> p.user.create, getUserByEmail -> p.user.findUnique). Reviewer: **[TEAM TO FILL: roll no./name]**

Sign-in Provider Strategy

Pattern type and category: Strategy — Behavioral (GoF)

Purpose and origin: authOptions.providers is a list of interchangeable provider objects behind one Provider type; NextAuth selects the provider by id at sign-in and delegates the authorization/profile mapping to it. Origin: framework (next-auth@4.22.1) configured by application.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Strategy	Provider = OAuthConfig | EmailConfig | CredentialsConfig	dependency next-auth@4.22.1 providers/index.d.ts:14
ConcreteStrategy	GoogleProvider({clientId, clientSecret})	`data/code/src/server/auth.ts:50-53`
Context	NextAuth(authOptions) with providers[]	`data/code/src/server/auth.ts:38-64`; `data/code/src/pages/api/auth/[...nextauth].ts:4`

Evidence: authOptions.providers is a list of interchangeable provider objects behind one Provider type; NextAuth selects the provider by id at sign-in and delegates the authorization/profile mapping to it. Only Google is configured; the commented block at auth.ts:54-62 documents adding more without changing NextAuth. Single concrete strategy, so variability is latent.

Diagram: `Part1/pattern_architecture/09_strategy_signin_provider.puml` (editable), `Part1/pattern_architecture/09_strategy_signin_provider.png`

Verification: Confirmed with the caveat of a single configured strategy (latent variability). Reviewer: **[TEAM TO FILL: roll no./name]**

TMDB Catalogue Facade

Pattern type and category: Facade — Structural (GoF)

Purpose and origin: Pages need categorised show lists, not URLs. Origin: application.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Facade	module lib/fetchers: getShows, getNewAndPopularShows, searchShows	`data/code/src/lib/fetchers.ts:5-77,82-122,124-142`
Subsystem	TMDB REST endpoints (trending, top_rated, discover, popular, search/multi) + API key	`data/code/src/lib/fetchers.ts:15-41`; `data/code/src/env.mjs`
Client	Home, MoviesPage, TVShowsPage	`data/code/src/app/(shows)/page.tsx:8-52`; `data/code/src/app/(shows)/movies/page.tsx:13-54`; `data/code/src/app/(shows)/tv-shows/page.tsx:12-51`
Client	NewAndPopularPage	`data/code/src/app/(shows)/new-and-popular/page.tsx:12-39`
Client	SiteHeader.searchShowsByQuery	`data/code/src/components/layouts/site-header.tsx:51-55`

Evidence: Pages need categorised show lists, not URLs. getShows(mediaType) issues eight TMDB requests in parallel, checks all responses, parses JSON and returns one object of named rows; callers only call getShows('movie'). URL construction, API key, network errors and JSON shape are hidden behind three functions (module-level facade, no class).

Diagram: `Part1/pattern_architecture/10_facade_tmdb_catalogue.puml` (editable), `Part1/pattern_architecture/10_facade_tmdb_catalogue.png`

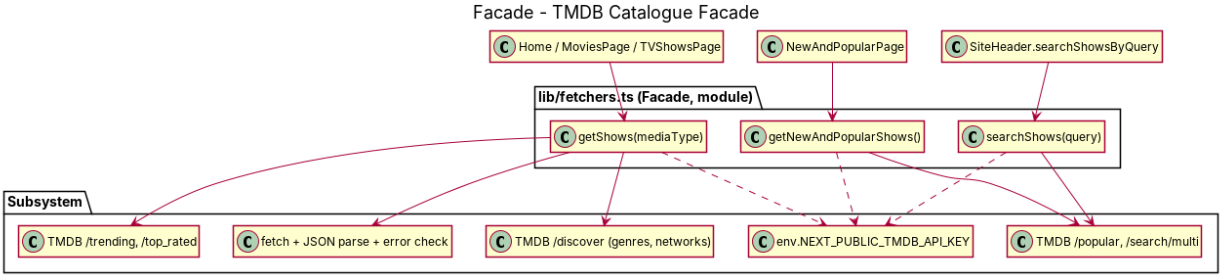


Figure 5: TMDB Catalogue Facade

Verification: Confirmed; facade behaviour pinned by characterization tests (URLs, order, keys, errors). Reviewer: **[TEAM TO FILL: roll no./name]**

API Router Tree

Pattern type and category: Composite — Structural (GoF); tRPC router tree

Purpose and origin: createTRPCRouter accepts a record whose values are procedures (leaves) or routers (composites); appRouter nests four routers that hold 19 procedures. Origin: application on framework (@trpc/server).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Component	AnyRouter | AnyProcedure in ProcedureRouterRecord	dependency @trpc/server@10.28.0
Composite (root)	appRouter	`data/code/src/server/api/root.ts:12-17`
Composite	userRouter	`data/code/src/server/api/routers/user.ts:9-91`
Composite	profileRouter	`data/code/src/server/api/routers/profile.ts:6-186`
Composite	iconRouter	`data/code/src/server/api/routers/icon.ts:5-59`
Composite	myListRouter	`data/code/src/server/api/routers/my-list.ts:6-184`
Leaf	19 procedures e.g. profile.getAll	`data/code/src/server/api/routers/profile.ts:7-15`
Factory	createTRPCRouter = t.router	`data/code/src/server/api/trpc.ts:97`

Evidence: createTRPCRouter accepts a record whose values are procedures (leaves) or routers (composites); appRouter nests four routers that hold 19 procedures. Clients address any node uniformly by dotted path (api.profile.getAll), and the type AppRouter is the whole tree. The recursive DecoratedProcedureRecord type (src/@trpc/next-layout/server/createTrpcNextLayout.tsx:53-65) walks routers and procedures the same way.

Diagram: `Part1/pattern_architecture/11_composite_api_router_tree.puml` (editable), `Part1/pattern_architecture/11_composite_api_router_tree.png`

Verification: Confirmed; uniform dotted-path access to routers and procedures; 19 leaves. Reviewer: **[TEAM TO FILL: roll no./name]**

Pages-Router tRPC Client Proxy

Pattern type and category: Proxy — Structural (GoF); remote proxy

Purpose and origin: Components call api.myList.getAll.useQuery(...) as if the server router were local. Origin: application on dependency (@trpc/next@10.28.0).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Subject	AppRouter type	`data/code/src/server/api/root.ts:20`
RealSubject	appRouter served by createNextApiHandler	`data/code/src/pages/api/trpc/[trpc].ts:8-19`
Proxy	api = createTRPCNext<AppRouter>(...)	`data/code/src/lib/api/api.ts:21-54`
Transport	client from the shared TRPCContext supplied by TRPCProvider (its own links in api.ts:36-45 are never executed)	`data/code/src/lib/api/client.ts:35-44`; `dependency@trpc/react-query@10.28.0dist/createHooksInternal-9f8a4b21.js:92,193`
Client	ShowsContainer, ShowModal, ProfilesScreen, SiteHeader and 7 more components	`data/code/src/components/shows-container.tsx:32-36`; `data/code/src/components/show-modal.tsx:93-124`

Evidence: Components call api.myList.getAll.useQuery(...) as if the server router were local. api is a JavaScript Proxy (createRecursiveProxy/createFlatProxy, @trpc/server dist index-4a823936.js:55,87) that turns each property path into an HTTP batch request to /api/trpc and returns typed results. Same typed interface as the real router (AppRouter).

Diagram: `Part1/pattern_architecture/12_proxy_pages_router_trpc_client.puml` (editable), `Part1/pattern_architecture/12_proxy_pages_router_trpc_client.png`

Verification: Confirmed; JS Proxy objects in @trpc/server dist (index-4a823936.js:55,87). **Corrected in the second pass:** its own links (api.ts:36-45) are never executed because createTRPCNext evaluates config() only inside withTRPC (@trpc/next@10.28.0 dist/index.js:171-188), which the app never mounts; its hooks use the client supplied through the shared TRPCContext by TRPCProvider (client.ts). Reviewer: **[TEAM TO FILL: roll no./name]**

App-Router tRPC Client Proxy

Pattern type and category: Proxy — Structural (GoF); remote proxy

Purpose and origin: A second client proxy built with the vendored createTRPCNextBeta returns createFlatProxy((key) => ...) (createTrpcNextBeta.tsx:76-92): 'Provider' and 'useContext' are served locally, every other key is decorated into React-Query hooks that call the server. Origin: application (vendored src/@trpc/next-layout) on dependency.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Subject	AppRouter type	`data/code/src/server/api/root.ts:20`
RealSubject	appRouter served by createNextApiHandler	`data/code/src/pages/api/trpc/[trpc].ts:8-19`
Proxy	api = createTRPCNextBeta<AppRouter>(...)	`data/code/src/lib/api/client.ts:22-45`
Proxy factory	createTRPCNextBeta -> createFlatProxy	`data/code/src/@trpc/next-layout/client/createTrpcNextBeta.tsx:48-93`
Provider	TRPCProvider wraps RootLayout	`data/code/src/context/trpc-provider.tsx:7-9`; `data/code/src/app/layout.tsx:80-96`
Client	AddProfileForm, EditProfileForm	`data/code/src/components/forms/add-profile-form.tsx:13`; `data/code/src/components/forms/edit-profile-form.tsx:15`

Evidence: A second client proxy built with the vendored createTRPCNextBeta returns createFlatProxy((key) => ...) (createTrpcNextBeta.tsx:76-92): 'Provider' and 'useContext' are served locally, every other key is decorated into React-Query hooks that call the server. It also supplies the QueryClient used by the app. Two independent proxies for one Subject is recorded as a smell (duplicated getBaseUrl/links).

Diagram: `Part1/pattern_architecture/13_proxy_app_router_trpc_client.puml` (editable), `Part1/pattern_architecture/13_proxy_app_router_trpc_client.png`

Verification: Confirmed; separate instance because it is a different proxy object built by vendored code; the duplication of two clients is noted as a smell. Reviewer: **[TEAM TO FILL: roll no./name]**

Environment Secret Guard

Pattern type and category: Proxy — Structural (GoF); protection proxy

Purpose and origin: Server secrets (STRIPE_API_KEY, DATABASE_URL, ...) must never be read in browser bundles. Origin: application (create-t3-app scaffold).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Subject	MergedOutput (validated env shape)	`data/code/src/env.mjs:67-71`
RealSubject	parsed.data (zod-validated variables)	`data/code/src/env.mjs:75-90`
Proxy	env = new Proxy(parsed.data, {get})	`data/code/src/env.mjs:92-105`
Client	10 modules importing env, e.g. lib/stripe.ts, lib/fetchers.ts, server/db.ts	`data/code/src/lib/stripe.ts:1-7`; `data/code/src/server/db.ts:3`

Evidence: Server secrets (STRIPE_API_KEY, DATABASE_URL, ...) must never be read in browser bundles. Every module imports 'env', which is a Proxy whose get trap checks the caller side: on the client any non NEXT_PUBLIC_ key throws a descriptive error, otherwise the value is forwarded from the validated object (env.mjs:93-103). Same interface as the real object, access control added transparently.

Diagram: `Part1/pattern_architecture/15_proxy_environment_secret_guard.puml` (editable), `Part1/pattern_architecture/15_proxy_environment_secret_guard.png`

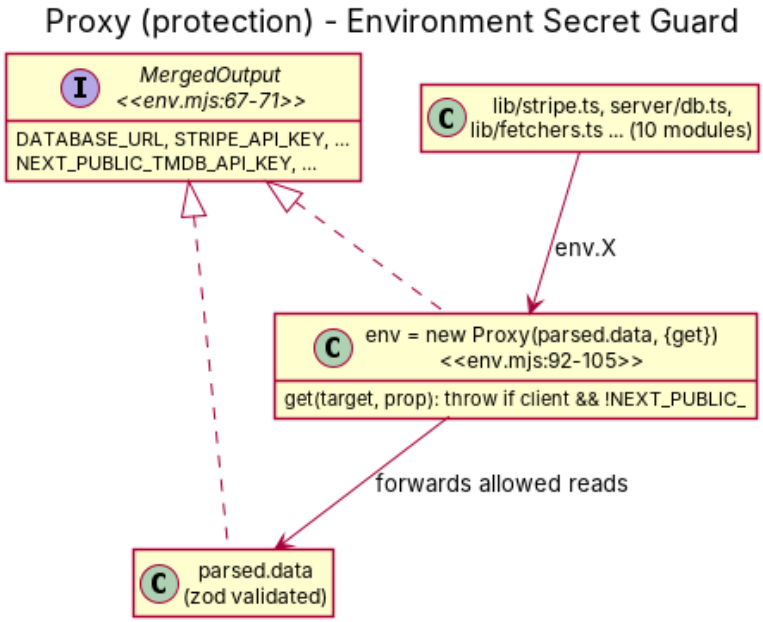


Figure 6: Environment Secret Guard

Verification: Confirmed (found in the second pass); same mechanism reproduced in Part 3 problem 09. Reviewer: **[TEAM TO FILL: roll no./name]**

My List Query Cache

Pattern type and category: Read-Through Cache — Architecture (caching); React Query

Purpose and origin: useQuery(key) returns cached data for the key (profile id) and only calls the loader on a miss or after invalidation; with staleTime Infinity, repeated renders of ShowsContainer and ShowModal share one fetch. Origin: application configuration on dependency (@tanstack/react-query@4.29.7).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Cache	QueryClient created once per TRPCProvider	`data/code/src/@trpc/next-layout/client/createTrpcNextBeta.tsx:55-63`
Cache policy	cacheTime: Infinity, staleTime: Infinity, refetchOnWindowFocus: false	`data/code/src/lib/api/client.ts:24-34`
Loader (data source)	myListRouter.getAll	`data/code/src/server/api/routers/my-list.ts:7-12`
Reader	ShowsContainer myShowsQuery	`data/code/src/components/shows-container.tsx:32-36`
Reader	ShowModal myShowsQuery	`data/code/src/components/show-modal.tsx:98-102`
Invalidator	ShowModal effect apiUtils.myList.getAll.invalidate()	`data/code/src/components/show-modal.tsx:127-130`

Evidence: useQuery(key) returns cached data for the key (profile id) and only calls the loader

on a miss or after invalidation; with staleTime Infinity, repeated renders of ShowsContainer and ShowModal share one fetch. After each add/remove mutation ShowModal invalidates the key so the next read reloads. Caller never talks to the data source directly.

Diagram: `Part1/pattern_architecture/14_cache_my_list_query_cache.puml` (editable), `Part1/pattern_architecture/14_cache_my_list_query_cache.png`

Verification: Confirmed; cache behaviour reproduced in Part 3 problem 10. The two client proxies share one React-Query context (verified in @trpc/react-query createHooksInternal:92,193 in the second pass). Reviewer: **[TEAM TO FILL: roll no./name]**

tRPC API Front Controller

Pattern type and category: Front Controller — Architectural (Fowler PoEAA); Next.js API route

Purpose and origin: Problem: 19 server operations would otherwise need 19 API route files, each repeating session lookup, body parsing and error formatting. Origin: application (create-t3-app scaffold) on framework (@trpc/server@10.28.0).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
FrontController	default export createNextApiHandler({router, createContext, onError}) at route /api/trpc/[trpc]	`data/code/src/pages/api/trpc/[trpc].ts:8-19`
Shared pre-processing	createTRPCContext (session lookup for every call)	`data/code/src/server/api/trpc.ts:60-69`
Shared error handling	onError logger (development only)	`data/code/src/pages/api/trpc/[trpc].ts:11-18`
Dispatcher	nodeHTTPRequestHandler -> resolveHTTPResponse -> callProcedure(path)	dependency @trpc/server@10.28.0 dist/nodeHTTPRequestHandler-5bbf93f1.js:7-34; dist/resolveHTTPResponse-9523b4e3.js:90
Command registry	appRouter (procedure paths user.*, profile.*, icon.*, myList.*)	`data/code/src/server/api/root.ts:12-17`
ConcreteCommand	19 procedures e.g. profile.getAll	`data/code/src/server/api/routers/profile.ts:7-15`
Client	httpBatchLink url /api/trpc	`data/code/src/lib/api/client.ts:41-43`

Evidence: Problem: 19 server operations would otherwise need 19 API route files, each repeating session lookup, body parsing and error formatting. The single Next.js dynamic route /api/trpc/[trpc] receives every call; createNextApiHandler reads the procedure path from req.query.trpc (adapters/next.js:17-25), builds the context once with createTRPCContext, and nodeHTTPRequestHandler/resolveHTTPResponse dispatch to the procedure registered under that path in appRouter (callProcedure, resolveHTTPResponse.js:90). Unknown paths produce a NOT_FOUND error shape from the same place, so dispatch, pre-processing and error policy are centralised.

Diagram: `Part1/pattern_architecture/16_front_controller_trpc_api.puml` (editable), `Part1/pattern_architecture/16_front_controller_trpc_api.png`

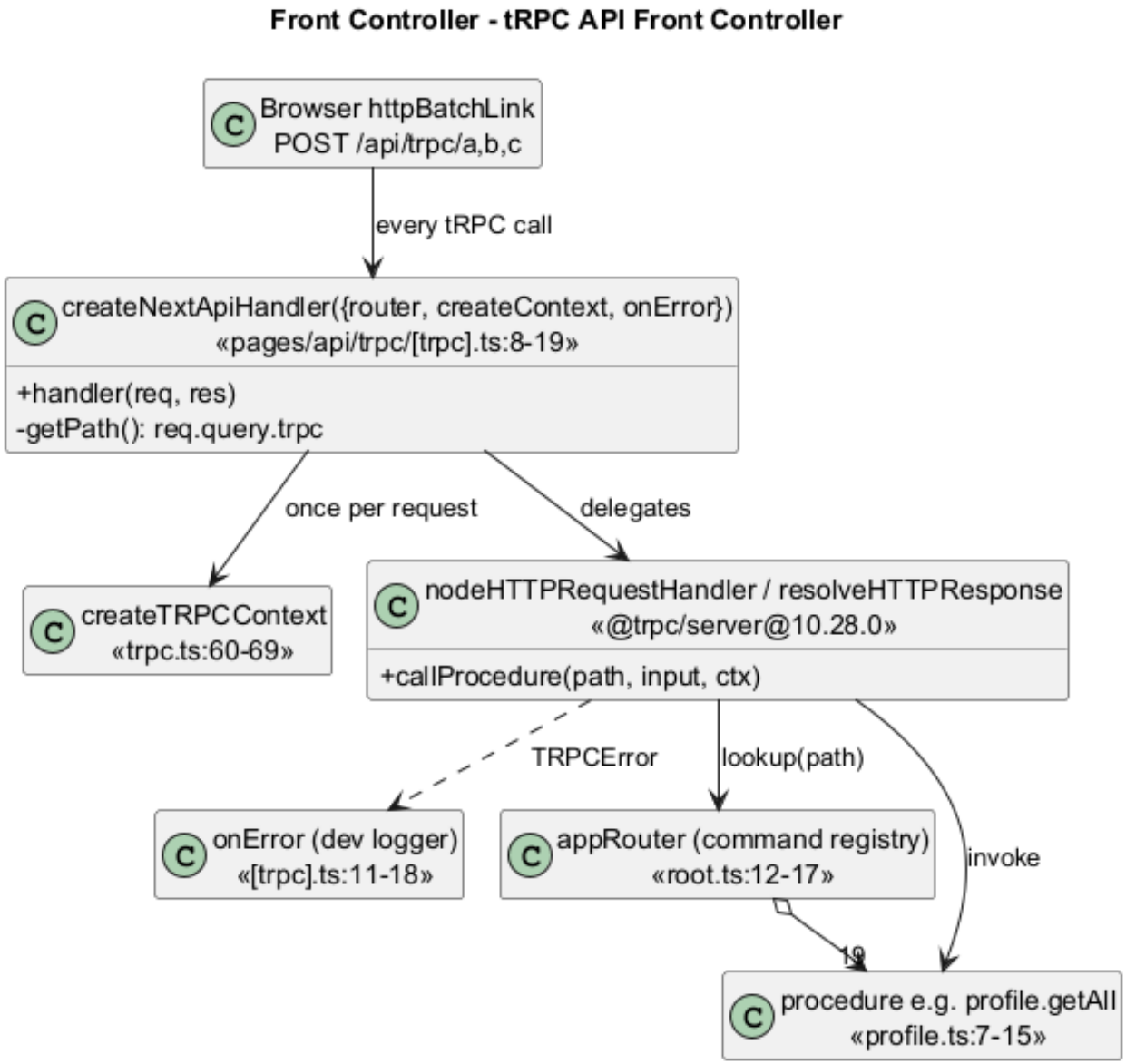


Figure 7: tRPC API Front Controller

Verification: Confirmed in the dependency source (adapters/next.js:16-56, resolveHTTPResponse.js:90). Reproduced in Part 3 problem 11 (14/14 cases incl. batching with one shared context). Alternative considered: Mediator — rejected, procedures do not interact through the controller. Reviewer: **[TEAM TO FILL: roll no./name]**

NextAuth Auth Route Front Controller

Pattern type and category: Front Controller — Architectural (Fowler PoEAA); NextAuth catch-all route

Purpose and origin: Every authentication request (GET /api/auth/providers, /session, /csrf, POST /signin/google, GET /callback/google, POST /signout) reaches the one handler exported by [...nextauth].ts. Origin: application on dependency (next-auth@4.22.1).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
FrontController	NextAuth(authOptions) exported at catch-all route /api/auth/[...nextauth]	`data/code/src/pages/api/auth/[...nextauth].ts:4`
Request parsing	NextAuthApiHandler maps req.query.nextauth -> {action, providerId}	dependency next-auth@4.22.1 next/index.js:18-31
Dispatcher	AuthHandler switch (action): providers, session, csrf, signin, signout, callback	dependency next-auth@4.22.1 core/index.js:68; core/index.js:159-206,257-340
Configuration	authOptions (adapter, providers, callbacks)	`data/code/src/server/auth.ts:38-64`
Client	LoginButton.loginWithGoogle -> signIn('google')	`data/code/src/components/login-button.tsx:13-24`
Client	SiteHeader sign-out -> signOut()	`data/code/src/components/layouts/site-header.tsx:230`

Evidence: Every authentication request (GET /api/auth/providers, /session, /csrf, POST /signin/google, GET /callback/google, POST /signout) reaches the one handler exported by [...nextauth].ts. next-auth splits the catch-all segment into action and providerId and AuthHandler dispatches with switch(action) to the matching route module, applying the shared authOptions (adapter, providers, session callback). The app router copy app/api/auth/[...nextauth]/_route.ts is disabled by its leading underscore, so this is the only active controller.

Diagram: `Part1/pattern_architecture/17_front_controller_nextauth.puml` (editable), `Part1/pattern_architecture/17_front_controller_nextauth.png`

Verification: Confirmed in next-auth core/index.js:159-206 (switch on action). The app-router copy _route.ts is inactive (underscore prefix). Reviewer: **[TEAM TO FILL: roll no./name]**

tRPC Next.js API Handler Adapter

Pattern type and category: Adapter — Structural (GoF); tRPC framework adapter

Purpose and origin: tRPC's router knows nothing about Next.js; Next.js only calls functions of the form (req, res). Origin: application wiring on dependency (@trpc/server@10.28.0 adapters/next).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Target	Next.js pages-router API handler signature (req: NextApiRequest, res: NextApiResponse)	dependency next@13.4.4 (NextApiHandler type)
Adapter	createNextApiHandler(opts) returns async (req, res) => ...	dependency @trpc/server@10.28.0 dist/adapters/next.js:16-56
Adaptee	tRPC request resolution nodeHTTPRequestHandler/resolveHTTPResponse over appRouter	dependency @trpc/server@10.28.0 dist/nodeHTTPRequestHandler-5bbf93f1.js:7-34
Adapted context type	CreateNextContextOptions used by createTRPCContext	`data/code/src/server/api/trpc.ts:28`; `data/code/src/server/api/trpc.ts:60-69`
Client	Next.js route file default export	`data/code/src/pages/api/trpc/[trpc].ts:8-19`

Evidence: tRPC's router knows nothing about Next.js; Next.js only calls functions of the form (req, res). createNextApiHandler converts between them: it extracts the path from Next's req.query.trpc (string or array), passes req/res into the framework-neutral nodeHTTPRequestHandler and writes the result back with res.json. The application only supplies appRouter and createTRPCContext typed with CreateNextContextOptions. Shares the route file with the tRPC API Front Controller but is a separate collaboration (interface conversion vs. central dispatch).

Diagram: `Part1/pattern_architecture/18_adapter_trpc_next_handler.puml` (editable), `Part1/pattern_architecture/18_adapter_trpc_next_handler.png`

Verification: Confirmed; separate collaboration from the Front Controller on the same line (interface conversion vs. central dispatch). Reviewer: **[TEAM TO FILL: roll no./name]**

tRPC Client Link Chain

Pattern type and category: Chain of Responsibility — Behavioral (GoF); tRPC client links

Purpose and origin: Each client operation is passed along an ordered list of links. Origin: application configuration on dependency (@trpc/client@10.28.0).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Handler	TRPCLink (op, next) => Observable	dependency @trpc/client@10.28.0
Chain runner	createChain execute(index) -> links[index]({op, next: execute(index + 1)})	dependency @trpc/client@10.28.0 dist/splitLink-aebd28df.js:4-23
ConcreteHandler	loggerLink({enabled}) - logs then forwards next(op)	`data/code/src/lib/api/client.ts:36-40`; `dependency@trpc/client@10.28.0dist/links/loggerLink.js:71-112`
ConcreteHandler (terminal)	httpBatchLink({url: /api/trpc}) - sends the batched HTTP request, never calls next	`data/code/src/lib/api/client.ts:41-43`; `dependency@trpc/client@10.28.0dist/links/httpBatchLink.js:130-224`
Chain assembly	links: [loggerLink, httpBatchLink] in createTRPCNextBeta options	`data/code/src/lib/api/client.ts:35-44`
Client	trpc.createClient(opts) inside TRPCProvider	`data/code/src/@trpc/next-layout/client/createTrpcNextBeta.tsx:55-63`

Evidence: Each client operation is passed along an ordered list of links. The chain runner calls links[0] with a next() that runs links[1]; loggerLink decides per operation whether to log (development or error results) and always forwards next(op); httpBatchLink is the terminal handler that performs the request. Adding/removing a link (e.g. a retry link) changes behaviour without touching callers. The identical links array in lib/api/api.ts:36-45 is never executed: createTRPCNext only reads config() inside withTRPC (dependency @trpc/next@10.28.0 dist/index.js:171-188), which the app never mounts, and its hooks use the shared TRPCContext supplied by TRPCProvider (dependency @trpc/react-query@10.28.0 createHooksInternal-9f8a4b21.js:92,193). That copy is therefore recorded as an uncertain/inactive observation, not a second instance.

Diagram: `Part1/pattern_architecture/19_cor_trpc_client_link_chain.puml` (editable), `Part1/pattern_architecture/19_cor_trpc_client_link_chain.png`

Verification: Confirmed for lib/api/client.ts; the identical chain in lib/api/api.ts is recorded as uncertain because it is never executed. Reviewer: **[TEAM TO FILL: roll no./name]**

tRPC Procedure Builder

Pattern type and category: Builder — Creational (GoF); fluent immutable builder (tRPC)

Purpose and origin: A procedure has many optional parts (input validators, output parser, meta, middleware stack, resolver kind). Origin: application on dependency (@trpc/server@10.28.0).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Builder	ProcedureBuilder returned by createBuilder (input, output, meta, use; build steps query/mutation/subscription)	dependency @trpc/server@10.28.0 dist/index.js:326-438
Builder factory	t = initTRPC.context<typeof createTRPCContext>().create({transformer, errorFormatter})	`data/code/src/server/api/trpc.ts:71-83`
ConcreteBuilder (base)	publicProcedure = t.procedure	`data/code/src/server/api/trpc.ts:106`
ConcreteBuilder (pre-configured)	protectedProcedure = t.procedure.use(enforceUserIsAuthed)	`data/code/src/server/api/trpc.ts:129`
Director / Client	router definitions, e.g. profile.getOthers = protectedProcedure.input(z.string()).query(resolver)	`data/code/src/server/api/routers/profile.ts:17-28`
Product	Procedure (createProcedureCaller(_def)) with merged inputs, middlewares and resolver	dependency @trpc/server@10.28.0 dist/index.js:412-438

Evidence: A procedure has many optional parts (input validators, output parser, meta, middleware stack, resolver kind). Instead of a large constructor, each builder call returns a NEW builder whose _def merges the previous definition (createNewBuilder concatenates inputs and middlewares, index.js:326-344), so shared partial builders such as protectedProcedure can be reused safely by all 18 private procedures. query()/mutation() are the final build steps that freeze the definition into a callable Procedure (createResolver -> createProcedureCaller).

Diagram: `Part1/pattern_architecture/20_builder_trpc_procedure_builder.puml` (editable), `Part1/pattern_architecture/20_builder_trpc_procedure_builder.png`

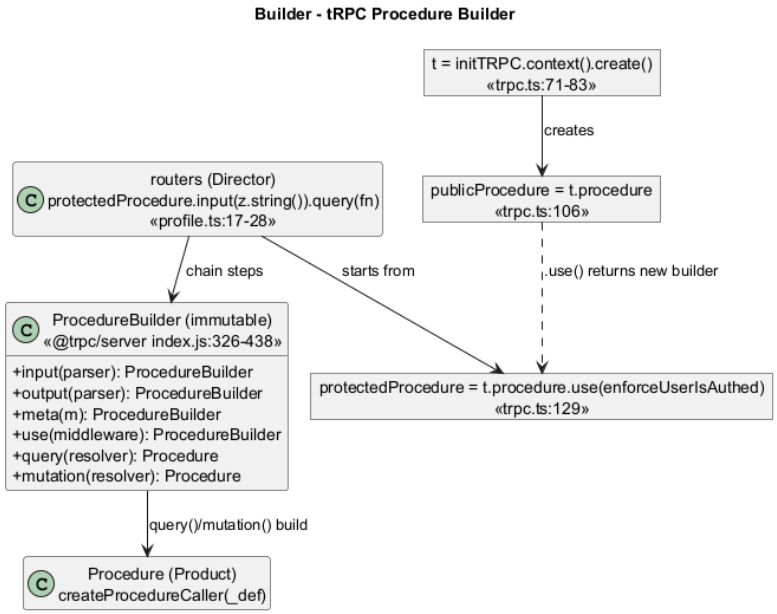


Figure 8: tRPC Procedure Builder

Verification: Confirmed in @trpc/server index.js:326-438 (createNewBuilder copies _def). Immutability reproduced in Part 3 problem 12 (case b11). Reviewer: **[TEAM TO FILL: roll**

no./name]

tRPC Request Context

Pattern type and category: Context Object — Architectural (Core J2EE); tRPC request context / dependency injection

Purpose and origin: Resolvers must not depend on Next.js req/res. Origin: application (create-t3-app scaffold).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
ContextFactory	createTRPCContext(opts: {req, res}) - resolves session per request	`data/code/src/server/api/trpc.ts:60-69`
Context builder (protocol independent)	createInnerTRPCContext({session}) -> {session, prisma}	`data/code/src/server/api/trpc.ts:47-52`
ContextObject type	CreateContextOptions / inferred ctx type bound by initTRPC.context<typeof createTRPCContext>()	`data/code/src/server/api/trpc.ts:33-35`; `data/code/src/server/api/trpc.ts:71`
Context refiner	enforceUserIsAuthed next({ctx: {session: non-null}})	`data/code/src/server/api/trpc.ts:109-119`
Consumer	resolvers reading ctx.prisma and ctx.session.user.id, e.g. profile.getAll	`data/code/src/server/api/routers/profile.ts:7-15`

Evidence: Resolvers must not depend on Next.js req/res. createTRPCContext extracts the protocol-specific data once (getServerAuthSession(req,res)) and hands a protocol-independent object {session, prisma} to every middleware and resolver via the ctx argument; createInnerTRPCContext exists so tests can build the same object without HTTP (comment trpc.ts:37-46). Middleware can return a refined context (enforceUserIsAuthed narrows session to non-null). Resolvers get their collaborators (database client, caller identity) only from ctx.

Diagram: `Part1/pattern_architecture/21_context_object_trpc_request.puml` (editable), `Part1/pattern_architecture/21_context_object_trpc_request.png`

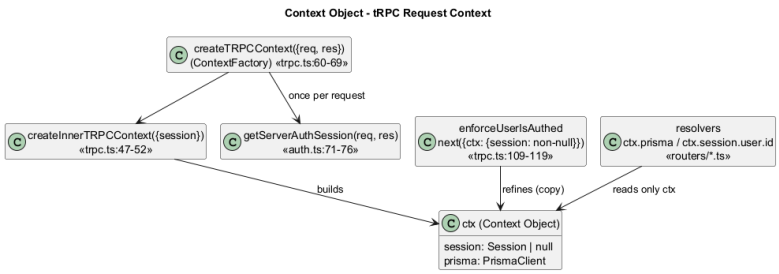


Figure 9: tRPC Request Context

Verification: Confirmed; the previously rejected Factory Method candidate on the same functions was reclassified as the ContextFactory role of this Context Object. Reproduced in Part 3 problem 13. Reviewer: **[TEAM TO FILL: roll no./name]**

Wire Serialization Transformer

Pattern type and category: Strategy — Behavioral (GoF); tRPC data transformer

Purpose and origin: How Date, Map or undefined values cross the HTTP boundary is an interchangeable algorithm. Origin: application configuration on dependency (@trpc/server@10.28.0, superjson@1.12.3).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Strategy	DataTransformer {serialize, deserialize} (normalised by getDataTransformer to input/output pair)	dependency @trpc/server@10.28.0 dist/config-f3ce0144.js:11-21
ConcreteStrategy (default)	defaultTransformer (identity) used when none is configured	dependency @trpc/server@10.28.0 dist/config-f3ce0144.js:22-30
ConcreteStrategy (selected)	superjson	dependency superjson@1.12.3; data/code/src/server/api/trpc.ts:30
Context (server)	initTRPC...create({transformer: superjson})	`data/code/src/server/api/trpc.ts:71-72`
Context (client)	createTRPCNextBeta({transformer: superjson})	`data/code/src/lib/api/client.ts:22-23`
Context (hydration)	createHydrateClient({transformer: superjson})	`data/code/src/lib/api/client.ts:51-53`

Evidence: How Date, Map or undefined values cross the HTTP boundary is an interchangeable algorithm. tRPC calls only transformer.serialize/deserialize; with no option it uses the identity defaultTransformer (JSON loses Date), the app selects superjson on both sides so a Date such as user.stripeCurrentPeriodEnd arrives as a Date. Swapping the strategy changes the wire format without touching any procedure. Server and client must choose the same strategy - a configuration constraint the pattern makes explicit.

Diagram: `Part1/pattern_architecture/22_strategy_wire_serialization.puml` (editable), `Part1/pattern_architecture/22_strategy_wire_serialization.png`

Verification: Confirmed with the caveat of one selected concrete strategy plus the framework default (identity). Reviewer: **[TEAM TO FILL: roll no./name]**

Search Input DOM Event Listeners

Pattern type and category: Observer — Behavioral (GoF); DOM EventTarget listeners

Purpose and origin: The search box must react to events that happen anywhere on the page. Origin: application on browser platform (EventTarget).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Subject	EventTarget addEventListener/removeEventListener (browser)	`browserDOMAPI`
ConcreteSubject	document (mousedown, touchstart) and window (keydown)	`data/code/src/hooks/use-on-click-outside.ts:15-16`; `data/code/src/components/debounced-input.tsx:69`
ConcreteObserver	useOnClickOutside listener (ignores clicks inside ref, else calls handler)	`data/code/src/hooks/use-on-click-outside.ts:8-14`
ConcreteObserver	DebouncedInput handleKeyDown (Escape closes, Ctrl/Cmd+K opens)	`data/code/src/components/debounced-input.tsx:53-71`
Subscription lifecycle	useEffect registers and its cleanup unregisters the listeners	`data/code/src/hooks/use-on-click-outside.ts:7-22`; `data/code/src/components/debounced-input.tsx:69-70`
Client	DebouncedInput uses useOnClickOutside(inputRef, closeInput); rendered by SiteHeader	`data/code/src/components/debounced-input.tsx:36-37`; `data/code/src/components/layouts/site-header.tsx:80`

Evidence: The search box must react to events that happen anywhere on the page. Instead of the page calling the input, the component subscribes listener objects to document/window; the browser notifies every registered listener on each event and the component unsubscribes on unmount (cleanup functions), so subject and observers stay decoupled and no listener leaks. Distinct from the Zustand observers: the subject is the browser EventTarget and the notification carries a DOM Event.

Diagram: `Part1/pattern_architecture/23_observer_search_input_dom_events.puml` (editable), `Part1/pattern_architecture/23_observer_search_input_dom_events.png`

Verification: Confirmed; subject is the browser EventTarget, listeners are unregistered in effect clean-ups. Reviewer: **[TEAM TO FILL: roll no./name]**

Shows Segment Error Boundary

Pattern type and category: Error Boundary — Framework idiom (React/Next.js App Router)

Purpose and origin: A failed TMDB call in a page must not blank the whole app. Origin: application on framework (next@13.4.4 error.tsx convention).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Boundary	ErrorBoundaryHandler (getDerivedStateFromError, reset) generated by Next for the (shows) segment	dependency next@13.4.4 dist/client/components/error-boundary.js:53-110
Fallback	Error page component (shows)/error.tsx	`data/code/src/app/(shows)/error.tsx:10-12`
Fallback view	ErrorScreen (message + Retry button calling reset())	`data/code/src/components/screens/error-screen.tsx:9-31`
Protected subtree	ShowsLayout children: Home, Movies, TV Shows, New and Popular, My List pages	`data/code/src/app/(shows)/layout.tsx:8-18`
Error source	getShows throws 'Failed to fetch shows' when a TMDB response is not ok	`data/code/src/lib/fetchers.ts:42-53`

Evidence: A failed TMDB call in a page must not blank the whole app. Next.js wraps the (shows) segment in an ErrorBoundary class component whose getDerivedStateFromError switches to the segment's error.tsx; the fallback renders ErrorScreen with the message and a Retry button bound to reset(), which clears the boundary state and re-renders the subtree. Header/layout outside the segment stay mounted.

Diagram: `Part1/pattern_architecture/24_error_boundary_shows_segment.puml` (editable), `Part1/pattern_architecture/24_error_boundary_shows_segment.png`

Verification: Confirmed in next@13.4.4 error-boundary.js:53-110; reproduced in Part 3 problem 14. Not observed at run time (no TMDB key in the analysis environment). Reviewer: **[TEAM TO FILL: roll no./name]**

Root Global Error Boundary

Pattern type and category: Error Boundary — Framework idiom (React/Next.js App Router)

Purpose and origin: Errors thrown by the root layout itself cannot be caught by segment boundaries. Origin: application on framework (next@13.4.4 global-error.tsx convention).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Boundary	ErrorBoundaryHandler wrapping the root layout	dependency next@13.4.4 dist/client/components/error-boundary.js:53-110
Fallback	GlobalError (renders its own <html><body>)	`data/code/src/app/global-error.tsx:10-18`
Fallback view	ErrorScreen	`data/code/src/components/screens/error-screen.tsx:9-31`
Protected subtree	RootLayout (TRPCProvider, html, body, Toast, Analytics)	`data/code/src/app/layout.tsx:78-98`

Evidence: Errors thrown by the root layout itself cannot be caught by segment boundaries. global-error.tsx replaces the whole document (it must render html/body because the root layout is gone) and reuses ErrorScreen with reset(). Independent of the shows boundary: different position in the tree, different fallback, catches different failures.

Diagram: `Part1/pattern_architecture/25_error_boundary_root_global.puml` (editable),

`Part1/pattern_architecture/25_error_boundary_root_global.png`

Verification: Confirmed as a separate boundary (different tree position and fallback). Reviewer: **[TEAM TO FILL: roll no./name]**

Shows Loading Placeholder

Pattern type and category: Suspense Boundary — Framework idiom (React Suspense / Next.js loading.tsx)

Purpose and origin: Async server pages suspend while getShows waits for eight TMDB requests. Origin: application on framework (next@13.4.4 loading.tsx convention).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Boundary	LoadingBoundary renders React.Suspense with the segment's loading component as fallback	dependency next@13.4.4 dist/client/components/layout-router.js:335-345
Fallback (placeholder)	ShowsLoading skeleton	`data/code/src/app/(shows)/loading.tsx:3-9`
Suspending child	async server pages awaiting TMDB, e.g. Home awaits getShows('movie')	`data/code/src/app/(shows)/page.tsx:8-52`

Evidence: Async server pages suspend while getShows waits for eight TMDB requests. Because (shows)/loading.tsx exists, Next wraps the segment in <Suspense fallback={<ShowsLoading/>}> so a skeleton is streamed immediately and replaced when the page resolves. The two explicit <Suspense> wrappers in account/users/[userId]/page.tsx and help-center/page.tsx are NOT counted: their children are client components that never suspend, so the fallback can never show (rejected candidate).

Diagram: `Part1/pattern_architecture/26_suspense_shows_loading_placeholder.puml` (editable), `Part1/pattern_architecture/26_suspense_shows_loading_placeholder.png`

Verification: Confirmed in next@13.4.4 layout-router.js:335-345; the explicit Suspense wrappers around client forms were rejected because their children never suspend. Reproduced in Part 3 problem 15. Reviewer: **[TEAM TO FILL: roll no./name]**

TMDB Row Scatter-Gather

Pattern type and category: Scatter-Gather — Concurrency / Enterprise Integration (Hohpe & Woolf)

Purpose and origin: A home page needs eight independent lists. Origin: application.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Scatter	Promise.all over 8 fetch calls (trending, top rated, Netflix originals, 5 genres)	`data/code/src/lib/fetchers.ts:6-40`
Gather / Aggregator	fail if any !res.ok, then Promise.all(res.json()) and build one categorized object	`data/code/src/lib/fetchers.ts:42-76`
Scatter	Promise.all over 4 fetch calls (popular/trending tv and movie)	`data/code/src/lib/fetchers.ts:83-97`
Gather / Aggregator	fail if any !res.ok, then merge into {popularTvs, popularMovies, trendingTvs, trendingMovies}	`data/code/src/lib/fetchers.ts:99-121`
Recipients	TMDB REST endpoints	`externalapi.themoviedb.org/3`
Client	Home / MoviesPage / TVShowsPage / NewAndPopularPage	`data/code/src/app/(shows)/page.tsx:8-52`; `data/code/src/app/(shows)/new-and-popular/page.tsx:12-39`

Evidence: A home page needs eight independent lists. getShows broadcasts all requests at once (Promise.all), waits for every reply, applies one all-or-nothing completeness rule (any non-ok response throws) and aggregates the replies into a single result keyed by row; getNewAndPopularShows does the same for four lists. Latency becomes the slowest call instead of the sum. Separate from the TMDB Catalogue Facade, which is about hiding endpoint details; this collaboration is about concurrent fan-out and aggregation.

Diagram: `Part1/pattern_architecture/27_scatter_gather_tmdb_rows.puml` (editable), `Part1/pattern_architecture/27_scatter_gather_tmdb_rows.png`

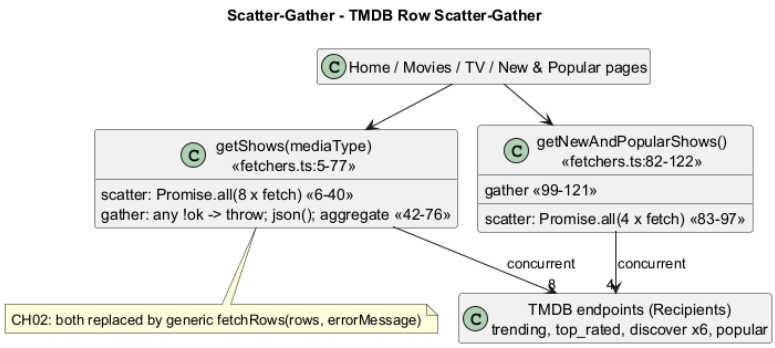


Figure 10: TMDB Row Scatter-Gather

Verification: Confirmed; separate from the Facade (fan-out and aggregation vs. hiding endpoints). Reproduced in Part 3 problem 16; modified by the CH02 refactoring. Reviewer: **[TEAM TO FILL: roll no./name]**

Profile and Account Query Cache

Pattern type and category: Read-Through Cache — Architecture (caching); React Query

Purpose and origin: Profiles and the current user are read by many screens; with staleTime Infinity each key (profile.getAll, profile.getOthers[id], profile.getOne[id], user.getCurrent) is fetched once and then served from the QueryClient. Origin: application configuration on dependency (@tanstack/react-query@4.29.7).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Cache	QueryClient created once per TRPCProvider (shared with My List Query Cache)	`data/code/src/@trpc/next-layout/client/createTrpcNextBeta.tsx:55-63`
Loader (data source)	profileRouter.getAll / getOthers / getOne; userRouter.getCurrent / getCurrentWithProfile	`data/code/src/server/api/routers/profile.ts:7-38`
Reader	ProfilesScreen profilesQuery	`data/code/src/components/screens/profiles-screen.tsx:29-31`
Reader	ManageProfiles profilesQuery	`data/code/src/components/manage-profiles.tsx:20`
Reader	SiteHeader otherProfilesQuery	`data/code/src/components/layouts/site-header.tsx:62-66`
Reader	AccountForm userQuery / IconPicker userQuery	`data/code/src/components/forms/account-form.tsx:81`; `data/code/src/components/icon-picker.tsx:31`
Invalidator	AddProfileForm onSuccess	`data/code/src/components/forms/add-profile-form.tsx:41-51`
Invalidator	EditProfileForm effect on useIsMutating()	`data/code/src/components/forms/edit-profile-form.tsx:88-91`
Invalidator	ResetPinForm onSuccess	`data/code/src/components/forms/reset-pin-form.tsx:44-63`
Invalidator	AccountForm onSuccess	`data/code/src/components/forms/account-form.tsx:245-255`
Invalidator	EditUserForm onSuccess	`data/code/src/components/forms/edit-user-form.tsx:40-48`

Evidence: Profiles and the current user are read by many screens; with staleTime Infinity each key (profile.getAll, profile.getOthers[id], profile.getOne[id], user.getCurrent) is fetched once and then served from the QueryClient. Every mutation that changes those records explicitly invalidates the affected keys (apiUtils.profile.getAll.invalidate() etc.), so the next read goes through to the loader. Independent of the My List Query Cache: different keys, loaders, readers and invalidation points; only the cache store is shared.

Diagram: `Part1/pattern_architecture/28_cache_profile_account_query.puml` (editable), `Part1/pattern_architecture/28_cache_profile_account_query.png`

Verification: Confirmed; independent of the My List cache except for the shared QueryClient. Reviewer: **[TEAM TO FILL: roll no./name]**

Rejected and uncertain candidates

Candidate	Decision	Reason
Caching Proxy for server components (`createTRPCNextLayout`)	rejected	Proxy + QueryClient structure exists (`src/@trpc/next-layout/server/createTrpcNextLayout.tsx:89-168`) but nothing imports it: dead code.

Candidate	Decision	Reason
Factory Method (`createTRPCContext`)	rejected	No creator hierarchy or overridable creation step; the same functions are the ContextFactory of the confirmed *tRPC Request Context* (Context Object).
Explicit Suspense boundaries	rejected	`account/users/[userId]/page.tsx:50-64`, `help-center/page.tsx:23-73` wrap client components that never suspend, so the fallback can never be shown.
Container / Presentational	rejected	`ShowsCarousel`/`ShowsGrid` read and write the modal store themselves, so the split is not maintained.
Mediator (Zustand stores)	rejected	Stores hold state and setters only, no coordination logic; already recorded as Observer subjects.
Command (`useMutation`)	rejected	Framework hook; no app-level command interface, queue, undo or logging.
Registry (`Icons`, `siteConfig`)	rejected	Static constant objects without a registration/lookup protocol.
Repository	rejected	Routers call `ctx.prisma.<model>` directly; no repository abstraction.
Template Method (nested layouts)	rejected	Layouts compose children; no base algorithm with overridable steps.
Recommendation strategy/engine	rejected	Fixed TMDB queries only (`app/(shows)/page.tsx:13-42`); no recommendation logic exists.
Debounce	rejected	UI timing technique (`components/debounced-input.tsx:44-50`), not a design pattern.
Observer via Stripe webhook	uncertain	Only the subscriber side is in code; registration lives in the Stripe dashboard; dispatch is an if-chain on `event.type`.
Protection Proxy for the PIN gate	uncertain	`ProfilesScreen` conditionally renders children; no common interface. The PIN is sent to the browser and compared there (`forms/pin-form.tsx:19`), so it is not real access control.
Implicit Next.js data cache	uncertain	`fetch` in server components is cached by Next 13 by default, but the code sets no option and it was not observed at run time.
Facade over session/subscription helpers	uncertain	`lib/session.ts:4-11`, `lib/subscription.ts:6-53` are thin single-subsystem wrappers.
Second client link chain (`lib/api/api.ts:36-45`)	uncertain	Declared like the active chain in `client.ts` but never executed (`withTRPC` not mounted).

3 Part 2 Evolution and refactoring

All three changes were implemented on branch `a12-evolution`. Before any change, commit `d68c773` added vitest and *characterization tests* that pin the baseline behaviour (no production code changed; 13/13 pass). Characterization tests added later for refactored code were first run on the untouched baseline (19/19, `data/test_result/part2/tests_baseline_characterization.log`). The four requirements of the case study are covered: *premium content* and the *new subscription plan* in CH01, *regional restrictions* in CH02, *parental control* in CH03. Patches: `data/test_result/part2/0001-*.patch` (test harness) and `Part2/changes/ChangeN/000*.patch`; evolved source: `Part2/code/`. The instances found in the Part 1 second pass were tracked through the same snapshots afterwards

(rebuilt by applying the patches to the upstream commit; final tree identical to `Part2/code/`).

CH01 — Premium content and a new subscription plan (Ultra)

Change and acceptance criteria: New plan **Ultra** (tier 5, 12.99/month, 4K+HDR, up to **7 profiles**). Titles of the originals row (TMDB network 213) are **premium** and playable only with Standard, Premium or Ultra (tier ≥ 3); other viewers see the title with an "Upgrade to watch" button. Acceptance (all automated in `tests/ch01-premium-plan.test.ts`): five plans incl. Ultra; UI and server plan lists identical; premium title denied for tiers 0–2 and allowed for 3–5; Basic subscriber denied, Standard allowed, anonymous `UNAUTHORIZED`; Ultra may create a 6th/7th profile but not an 8th, Standard keeps 5; only the originals row tagged premium; all earlier tests pass.

Before and after: commits `d68c773` → `3f79642` (22 files, +642/-186 incl. tests); diff `Part2/changes/Change1/change.diff`, file list `Part2/changes/Change1/diffstat.txt`, requirement record `Part2/changes/Change1/README.md`.

Implementation and tests: New `config/plan-catalog.ts`; `config/plans.ts` and `config/subscriptions.ts` derived from it; `lib/access/*` (`AccessRule`, `PremiumContentRule`); `server/api/trpc.ts` (`attachSubscription`, `subscribedProcedure`); new `routers/catalogue.ts`; `routers/profile.ts` limit from `ctx.maxProfiles`; `lib/fetchers.ts` tags premium; `components/show-modal.tsx` upgrade button; checkout refuses a plan without Stripe price; env `STRIPE_ULTRA_PRICE_ID`. Test fixture change disclosed: fake `user.findFirst` returns a planless user (a signed-in user always has a row); no assertion changed. Run: `yarn test` and `yarn typecheck` in `Part2/code`. **Actual result: 29/29 tests passed, typecheck pass** (`Part2/changes/Change1/test_result.log`).

Pattern effects: Authenticated Procedure Pipeline **extended** (new handler `attachSubscription`); API Router Tree **extended** (`catalogueRouter`); TMDB Catalogue Facade **modified** (premium tagging); tRPC API Front Controller **extended** (new command `catalogue.checkAccess`); tRPC Procedure Builder **extended** (`subscribedProcedure`; `protectedProcedure` unchanged thanks to immutable builders); tRPC Request Context **extended** (plan entitlements); **newly introduced**: Content Access Rule Chain (CoR) and Access Rule Evaluation Template (Template Method: `AccessRule.check` fixed, `evaluate` abstract); all others preserved.

Refactoring: *Shotgun Surgery / duplicated data*: plans existed in `config/plans.ts` (UI) and `config/subscriptions.ts` (server) and had already drifted (Basic devices "Phone, Tablet, Computer" vs "…, TV"). Root cause: the server list reads secret price ids through the env Protection Proxy, which throws in the browser. Technique: extract a client-safe plan catalogue and derive both lists; *Replace Magic Number* 5 with `plan.maxProfiles` (default 5). Behaviour preservation: subscription and profile-limit characterization tests unchanged and passing; a new test makes future drift fail.

Before/after metrics: lizard 1.24.1, scope `src/` without tests and CSS, identical for every snapshot (`data/test_result/part2/run_snapshots.sh`).

Measure	baseline	CH01
src files / NLOC / functions	110 / 6694 / 280	115 / 6829 / 297
average CCN	2.31	2.35
plan data (plans.ts + subscriptions.ts + plan-catalog.ts): NLOC / max CCN	139 / 0	123 / 3
tRPC pipeline (server/api/trpc.ts): NLOC / max CCN	49 / 3	63 / 6
tests passed	0 (13/13 at d68c773)	29/29

Max CCN of the whole `src/` is reported as 23 → 25 by lizard; this is a **parser artefact**: the

CCN-25 function is the unchanged "Add to My List" `onClick` handler in `show-modal.tsx` (body identical, checked by script), which lizard did not detect as a function in the baseline JSX.

CH02 — Regional restrictions

Change and acceptance criteria: Titles are licensed per territory (`allowedRegions` or `blockedRegions`); region from the `x-vercel-ip-country` header, else `cf-ipcountry`, else US. Unlicensed titles must not appear in any row and must be refused on direct access; a region-blocked premium title is hidden, not upsold. Acceptance (`tests/ch02-region.test.ts`): region normalisation and header priority; tRPC context reads the header; Fight Club (550) hidden in CN, allowed in US; Game of Thrones (1399) allowed in IN, hidden in DE; blocked premium title gives `REGION_BLOCKED`; locked titles stay visible, hidden ones are removed; row titles and order unchanged; `checkAccess` uses the request region.

Before and after: commits `3f79642` → `e420dcf` (17 files, +499/-231 incl. tests); diff `Part2/changes/Change2/change.diff`, file list `Part2/changes/Change2/diffstat.txt`, requirement record `Part2/changes/Change2/README.md`.

Implementation and tests: `config/regions.ts`, `lib/region.ts`, `lib/server-region.ts`; `RegionRule` placed first in the chain, `visibleShows`; region in the tRPC context; `lib/catalogue.ts` + the four `(shows)` pages filter rows on the server. Run: `yarn test` and `yarn typecheck` in `Part2/code`. **Actual result: 45/45 tests passed, typecheck pass** (`Part2/changes/Change2/test_result.log`).

Pattern effects: Content Access Rule Chain **extended** (RegionRule linked at the front); Access Rule Evaluation Template **extended**; TMDB Catalogue Facade **modified** (internal refactoring, interface preserved); TMDB Row Scatter-Gather **modified** (two hand-written fan-outs replaced by the generic `fetchRows` aggregator); tRPC Request Context **extended** (`region`); others preserved.

Refactoring: (1) *Duplicate Code / Long Method* in `lib/fetchers.ts`: 8 + 4 copies of fetch / ok-check / json / return field and an 8-term `||` condition. Technique: rows as data + one `fetchRows` helper (Extract Function). (2) *Duplicate Code*: the same 7-entry title/row array in the Home, Movies and TV pages. Technique: `GENRE_ROWS` + `toCategories` (Extract Function). Behaviour preservation: the URL/order/key/error characterization tests pass on the original and on CH02; row titles pinned (the TV page keeps "Scary Movies").

Before/after metrics: lizard 1.24.1, scope `src/` without tests and CSS, identical for every snapshot (`data/test_result/part2/run_snapshots.sh`).

Measure	CH01	CH02
src files / NLOC / functions	115 / 6829 / 297	119 / 6845 / 322
average CCN	2.35	2.25
TMDB facade (lib/fetchers.ts): NLOC / max CCN	126 / 9	120 / 3
row pages (4 shows pages + lib/catalogue.ts): NLOC / max CCN	174 / 3	116 / 3
tests passed	29	45/45

CH03 — Parental control

Change and acceptance criteria: Each profile has a maturity rating **KIDS / TEEN / ADULT** (default ADULT, so existing profiles are unchanged), set on Edit Profile. KIDS hides adult titles and Horror, Thriller, Crime, War; TEEN hides adult titles and Horror. Restricted titles disappear from rows, hero, search and My List of that profile; the server refuses them in `catalogue.checkAccess`

and `myList.create`; another account's profile id is refused. Acceptance (`tests/ch03-parental.test.ts`): 9 rating cases; chain order region → maturity → premium; client filter only for restricted profiles; foreign profile `FORBIDDEN`; Kids profile cannot save horror; rating stored by `profile.update`.

Before and after: commits `e420dcf` → `f393031` (19 files, +536/-147 incl. tests); diff `Part2/changes/Change3/change.diff`, file list `Part2/changes/Change3/diffstat.txt`, requirement record `Part2/changes/Change3/README.md`.

Implementation and tests: `enum MATURITY` + `Profile.maturityLevel` in `prisma/schema.prisma` (apply with `prisma db push`); `config/maturity.ts`; `MaturityRule` between Region and Premium; `profileVisibleShows`; `catalogue.checkAccess` loads the profile with an owner check; maturity guard in `myList.create`; rating selector in `edit-profile-form.tsx`; filtering in `ShowsContainer`, `Hero`, search grid. Run: `yarn test` and `yarn typecheck` in `Part2/code`. **Actual result: 65/65 tests passed, typecheck pass** (`Part2/changes/Change3/test_result.log`).

Pattern effects: Content Access Rule Chain and Access Rule Evaluation Template **extended** (MaturityRule); Active Profile Selection **extended** (Hero becomes a new observer, selector on `maturityLevel`); Search Results Broadcast **modified** (ShowsContainer filters the broadcast results); Persisted Profile Store preserved (persisted profile now carries the rating); all 13 second-pass instances preserved; others preserved.

Refactoring: *Duplicate Code* in `routers/my-list.ts`: identical 17-field zod schema twice, the "load profile or NOT_FOUND" block three times, field-by-field copies twice. Technique: Extract Variable (`showInput`), Extract Function (`findProfileWithList`, `findShowOrThrow`, `showData`). Behaviour preservation: `tests/characterization-mylist.test.ts` (update/delete errors and written data) ran on the original first and passes on CH03.

Before/after metrics: lizard 1.24.1, scope `src/` without tests and CSS, identical for every snapshot (`data/test_result/part2/run_snapshots.sh`).

Measure	CH02	CH03
src files / NLOC / functions	119 / 6845 / 322	120 / 6916 / 328
average CCN	2.25	2.26
my-list router (routers/my-list.ts): NLOC / max CCN	169 / 3	114 / 3
access chain (lib/access/*): NLOC / max CCN	116 / 7	168 / 7
tests passed	45	65/65

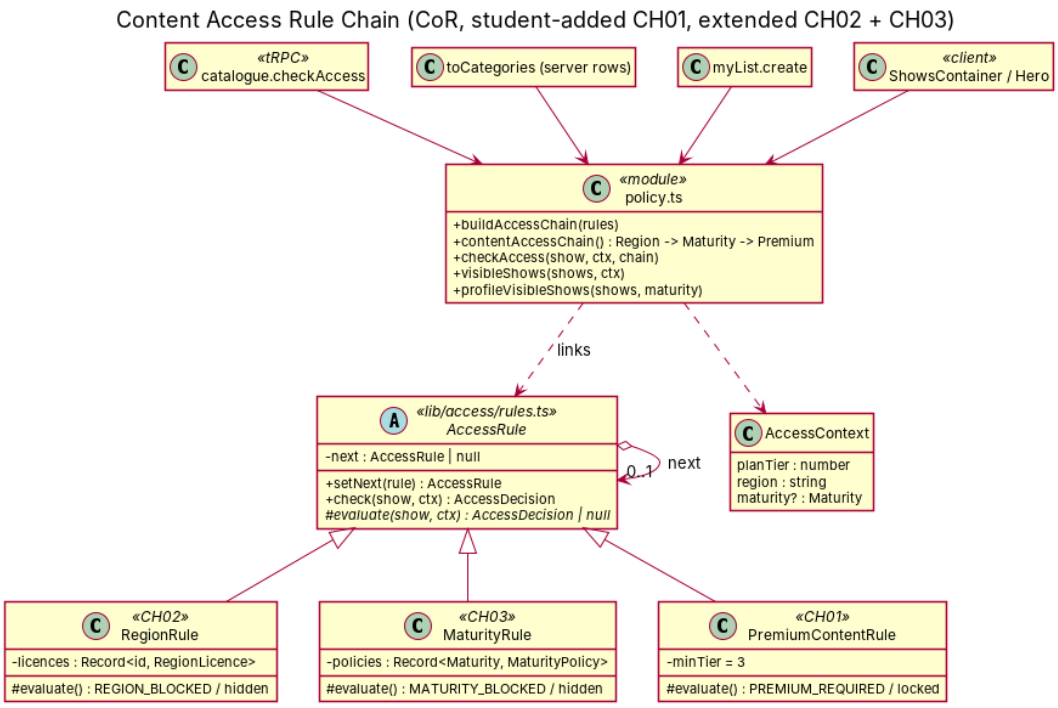


Figure 11: Newly introduced Content Access Rule Chain after CH03 (`Part2/diagrams/01_content_access_rule_chain_CH03.puml`).

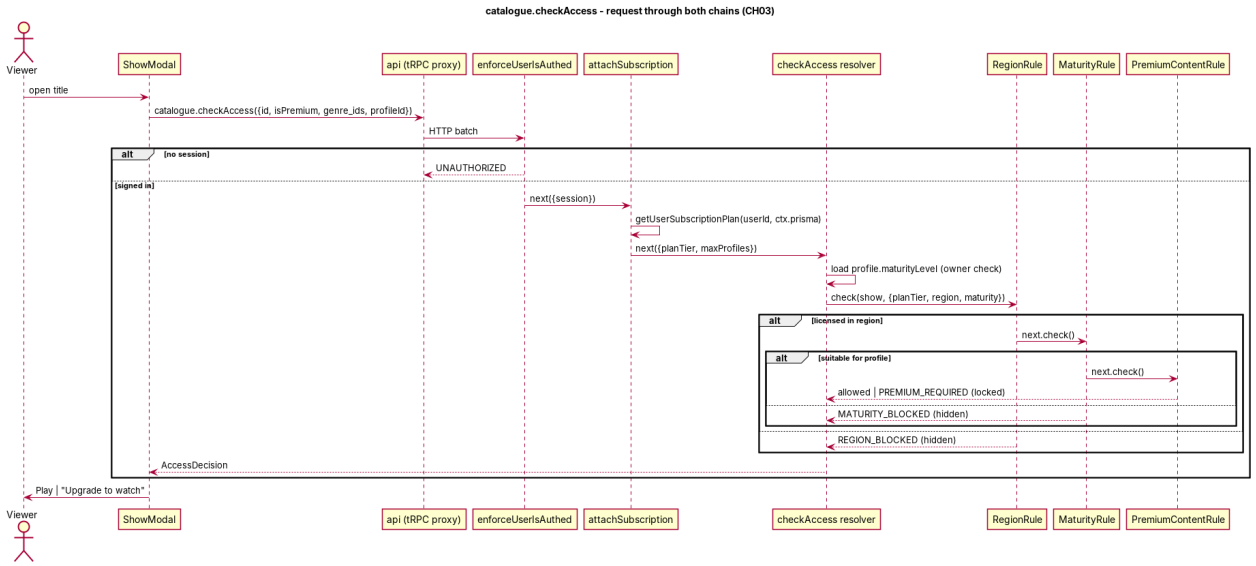


Figure 12: One `catalogue.checkAccess` request through both chains (auth middleware chain and access rule chain).

Further updated UML: `Part2/diagrams/03_template_method_access_rule_CH03`, `04_cor_authenticated_pipeline_CH01`, `05_composite_api_router_tree_CH01`, `06_observer_active_profile_selection_CH03`, `07_facade_tmdb_refactored_CH02`, `08_plan_catalogue_refactor_CH01`, and for the second-pass instances `09_front_controller_trpc_api_CH01`, `10_builder_procedure_builder_CH01`, `11_context_object_request_context_CH02`, `12_scatter_gather_fetch_rows_CH02` (`.puml` + `.png`).

3.1 Updated pattern counts

Counts per snapshot (`Part2/Pattern_count.csv`); rows with participants, locations, status and change description per instance and snapshot in `Part2/pattern_evalution.csv` (118 rows; all 374 evolved locations computed from anchors per snapshot and checked, 0 problems).

Snapshot	Change in counts	Types	Instances	Concrete	Rejected / uncertain
baseline 676e5dd	—	16	28	50	11 / 5
CH01 3f79642	+ Content Access Rule Chain (CoR), + Access Rule Evaluation Template (new type Template Method); Front Controller, Builder, Context Object extended	17	30	56	11 / 5
CH02 e420dcf	instances continue (RegionRule added; Scatter-Gather modified; region in the context)	17	30	57	11 / 5
CH03 f393031	instances continue (MaturityRule, Hero observer)	17	30	60	11 / 5

Per-type instance counts after CH01–CH03: Observer 4, Chain of Responsibility 3, Proxy 3, Singleton 2, Adapter 2, Strategy 2, Read-Through Cache 2, Front Controller 2, Error Boundary 2, and Composite, Decorator, Facade, Builder, Context Object, Suspense Boundary, Scatter-Gather, Template Method 1 each. (Concrete counts: baseline from the Part 1 records, snapshots computed from the role definitions in `data/test_result/part2/part2_evolution.py`.) No instance was replaced or removed. The two new instances are student-added and share participant classes: the *chain* is the object-level collaboration (links via `setNext`), the *template* is the class-level one (`check` fixed in the base class, `evaluate` overridden). The old Authenticated Procedure Pipeline continues under its name and now feeds the plan tier into the new chain. **Interpretation: Benefits.** Each new rule was one class plus one line in `contentAccessChain`; the CH02/CH03 diffs did not touch `PremiumContentRule`. Ordering "hide before lock" is explicit and tested. Extending the existing tRPC chain (`attachSubscription`) gave every resolver the plan once per request; because tRPC builders are immutable, deriving `subscribedProcedure` could not change the 18 existing protected procedures, and the Context Object absorbed both the plan (CH01) and the region (CH02) without touching resolver signatures. The refactorings removed real duplication: the facade's max CCN fell $9 \to 3$, the row pages $174 \to 116$ NLOC, the My-List router $169 \to 114$ NLOC, and the plan data shrank $139 \to 123$ NLOC while gaining a fifth plan. **Trade-offs.** Total NLOC grew $6694 \to 6916$ (+3.3%) and functions $280 \to 328$, mostly new features and small helpers; more LOC here is not worse design and less LOC alone would not prove better design. The rule chain is evaluated in three places (server rows, tRPC, browser) with different context completeness (rows know region only, the browser knows maturity only). **Remaining limitations.** Sample licence/premium data; hand-written profile `select` in four pages (another Shotgun Surgery spot); the persisted profile keeps an old rating until re-selected; pre-existing missing ownership checks on profile/My-List procedures; no end-to-end run with real services.

4 Part 3 Cross-language comparison

Selected instances: One verified instance per confirmed baseline type (16 problems; problems 11–16 were added for the six types found in the second pass). Where a type has several instances we chose the one that best shows the pattern in a small problem: Chain of Responsibility — *Content Access Rule Chain* (CH03 state; richer than the one-handler auth pipeline and also exercises the Template Method introduced in CH01); Singleton — *Shared Prisma Database Client* (explicit holder, hot-reload behaviour); Observer — *Active Profile Selection* (most observers, extended in CH03); Proxy — *Environment Secret Guard* (protection logic shows the language mechanisms best); Decorator — Persisted Profile Store; Adapter — NextAuth Prisma Persistence Adapter; Strategy — Sign-in Provider Strategy; Facade — TMDB Catalogue Facade (after CH02); Composite — API Router Tree (after CH01); Read-Through Cache — My List Query Cache; Front Controller — *tRPC API Front Controller* (after CH01; richer than the NextAuth controller and shows batching with one shared context); Builder — tRPC Procedure Builder; Context Object — tRPC Request Context (after CH02, with region and plan refinement); Error Boundary — Shows Segment Error Boundary, modelled together with the Root Global Error Boundary because their interaction (fallback errors propagate upwards) is the essential behaviour; Suspense Boundary — Shows Loading Placeholder; Scatter-Gather — TMDB Row Scatter-Gather (after CH02).

Common specifications: For each problem, `Part3/<problem>/spec.md` gives inputs, outputs, rules and error cases, and `Part3/<problem>/cases.tsv` is the *single shared fixture* read by all four test runners (8–16 cases each, 177 in total, including error cases such as unknown maturity, invalid NODE_ENV, unknown decorator, ORM constraint errors, unsupported provider, invalid media type, server-variable access on the client, missing My-List entry, unknown procedure path, missing input, unauthenticated call, building twice, invalid geo header, failing fallback, failing fetch). Test doubles that belong to the problem (fake HTTP client in 07) are defined identically in every runner.

Implementations: Folders `Part3/NN_<pattern>_<problem>/{java,python,javascript,cpp}/` with production code, a separate test runner and a README with the run command. Runtimes: javac 21.0.12.1 / openjdk version "21.0.12.1" 2026-08-18 LTS; Python 3.12.10; Node.js v26.13.0; g++ (MinGW-W64 x86_64-ucrt-posix-seh, built by Brecht Sanders, r3) 16.2.0 -std=c++17 -O2. Commands (working folder = language folder):

- Java: `javac -d build *.java`; `java -cp build Runner ../cases.tsv`
- Python: `python -B test_runner.py ../cases.tsv`
- JavaScript: `node testRunner.js ../cases.tsv`
- C++: `g++ -std=c++17 -O2 -Wall -o build/test_runner test_runner.cpp`; `./build/test_runner ../cases.tsv`

All 64 are built, run, timed and measured by `data/test_result/part3/run_all.py` (run 2026-10-30 on Windows 11 x64; toolchain in `data/test_result/part3/environment.txt`); the script also regenerates every language README from the measured values.

Actual results: All 64 implementations pass all their cases (177 cases × 4 languages = 708 checks); none marked not run. Logs: `Part3/<problem>/<language>/test_result.log`.

Static and dynamic comparison: Rules: LOC = lizard NLOC of production files only; max CC = largest lizard cyclomatic complexity of one function; time = median of 5 runs after 1 warm-up, whole process incl. runtime start-up and reading the fixture (raw: `data/test_result/part3/timings.csv`). Full table with notes: `Part3/llm/pattern_crosslanguage.csv`.

	source LOC				median ms			
Problem	Java	Py	JS	C++	Java	Py	JS	C++
Chain of Responsibility (01)	69	68	60	86	108.2	100.7	54.1	14.7

	source LOC				median ms			
Problem	Java	Py	JS	C++	Java	Py	JS	C++
Singleton (02)	28	15	17	28	92.1	39.1	51.5	14.7
Observer (03)	44	30	24	50	94.0	82.8	49.1	13.7
Decorator (04)	71	37	34	59	108.4	44.6	53.3	15.3
Adapter (05)	50	37	31	59	124.1	46.9	58.0	18.3
Strategy (06)	55	29	31	63	113.3	68.6	59.3	15.7
Facade (07)	49	25	34	62	120.2	47.1	59.0	15.9
Composite (08)	68	36	36	70	101.4	42.9	53.4	14.9
Proxy (09)	26	11	16	39	104.9	41.8	56.0	16.5
Read-Through Cache (10)	37	28	33	42	108.8	85.1	65.0	15.4
Front Controller (11)	68	49	58	74	154.5	48.3	59.7	17.7
Builder (12)	94	66	81	104	134.0	48.9	59.6	21.6
Context Object (13)	78	40	50	79	130.6	87.2	67.2	20.4
Error Boundary (14)	76	41	57	84	122.9	58.1	68.1	19.7
Suspense Boundary (15)	60	41	51	75	123.5	51.3	60.9	19.9
Scatter-Gather (16)	26	15	10	45	136.0	267.9	66.9	18.4
Total LOC / median of medians	899	568	623	1019	116.8	50.1	59.1	16.2
Max function CC (all problems)	11	9	10	11				

Comparison: Participant structures. The same roles appear in every language, but how they are expressed differs. Java and C++ name every role as a type (interfaces / abstract classes, `final`/`override`), so the participant mapping is visible in declarations; Python relies on duck typing (ABC only where an abstract step must be enforced, e.g. `AccessRule.evaluate`); JavaScript often needs no class at all: observers are functions in a `Set`, decorators are functions `store` → `store` that spread and override, the adapter is a factory returning an object literal — exactly the style of zustand, next-auth and tRPC in the source. **Abstraction and inheritance vs. composition.** Template Method needs inheritance in all four (abstract `check`/`evaluate`); all other problems use composition: decorators wrap, chains link, the facade holds the HTTP client. In C++ ownership is part of the design (`unique_ptr` for chain links and decorators, non-owning pointers for observers, `shared_ptr` to keep the singleton's identity across reloads). **Language features.** Proxy is the clearest difference: JavaScript's built-in `Proxy` traps property access (identical to `env.mjs`); Python's `__getattr__` intercepts attribute reads; Java's `java.lang.reflect.Proxy` intercepts interface calls at run time via reflection; C++ has no run-time interception, so the proxy is an ordinary class implementing the same interface. Records/dataclasses/structs give value objects; `Optional`/`std::optional`/`None`/`null` express "no result" in the chain and adapter. JavaScript is the only version of the facade that is asynchronous (`Promise.all`), like the source. **Static vs. dynamic results.** Python and JavaScript need less code than Java and C++ (568 and 623 vs. 899 and 1019 NLOC in total over the 16 problems); in 59 of 64 implementations no function exceeds CC 7; the exceptions are the Front Controller dispatch methods and the C++ Builder director (CC 9–11). Measured times (Windows 11 x64) are dominated by process start-up: native C++ 13.7–21.6 ms, Python 39.1–267.9 ms (the maximum is Scatter-Gather, which starts an asyncio event loop per case), Node.js 49.1–68.1 ms, JVM 92.1–154.5 ms. The timing therefore compares runtimes, not the patterns. New problems: the Front Controller and the C++ Builder director have the highest function complexity (CC 9–11: one branch per error code / build step), Scatter-Gather is the smallest in JavaScript (10 NLOC) because `Promise.all` *is* the pattern there, and the C++

Scatter-Gather is the only version that starts real OS threads (`std::async`).

Technology discussion: The case-study stack implements several patterns in its frameworks, so the application inherits them: tRPC middlewares form a Chain of Responsibility in which each middleware calls `next()` or stops the request [8]; zustand middlewares (`persist`, `devtools`) are decorators over a state creator [9]; NextAuth separates providers (strategies) and database adapters [10]; TanStack Query is a read-through cache with explicit invalidation [11]. JavaScript's `Proxy` [5] is used both by tRPC (remote proxy) and by the T3 `env` module (protection proxy), while Java offers dynamic proxies only for interfaces [6] and Python offers attribute hooks [7]. This matches the GoF observation that patterns depend on language features [1]: in a language with first-class functions and proxies several GoF patterns shrink to idioms, which is what the JavaScript column and the source code show.

5 LLM records and human verification

Run and model: Session 1: Claude (claude.ai), displayed model **Claude Opus 5.5**, access tier **[TEAM TO FILL: Free / Pro]**, 2026-10-10, Parts 1–3 (PROMPT 1 in each prompt file). Session 2: Claude Code desktop app, displayed model **Claude Opus 5.5**, 2026-10-29/30, exhaustive second pass of Part 1, evolution of the new instances, Part 3 problems 11–16 and the full re-run (PROMPT 2). Prompt files: `Part1/Part1_prompt.txt`, `Part2/Part2_prompt.text`, `Part3/llm/part3_prompt.txt`; records: `Part1/llm.csv`, `Part2/llm.csv`, `Part3/llm/llm.csv`. The model worked in a sandbox with a shell: it cloned the application, ran the typechecker, tests, lizard and the Part 3 programs itself, so its numeric results come from real runs. **[TEAM TO FILL: if the course requires a free tool, repeat the Part 1 prompt with a free model and add a row to Part1/llm.csv]**

Inputs and outputs: Input: `data/code/` (commit 676e5dd), the three assignment files and (session 2) the dependency sources of the pinned versions. Outputs and claim tables: `data/test_result/llm/part1_llm_output.md`, `part2_llm_output.md`, `part3_llm_output.md` (session 1) and `part1_second_pass_output.md`, `part2_second_pass_output.md`, `part3_second_pass_output.md` (session 2), all in `data/test_result/llm/`. The raw transcript must be exported from claude.ai into the same folders **[TEAM TO FILL: export transcript]**.

Claims and evaluation: Partially correct, corrected before submission. Errors found by checking the model's own work against the source and tools: (1) first inventory had 14 instances — the Environment Secret Guard proxy was missed and added on a second pass; (2) three line ranges were wrong (two end lines, one offset) — found by an independent locator and corrected; (3) first Part 2 NLOC run ignored `.tsx` files (1617 instead of 6694) — corrected; (4) a reported max-CCN increase was identified as a lizard parsing artefact. One claim was not accepted: the Stripe webhook as Observer (kept as uncertain). Session 2: 16 Part 1 claims checked against application and dependency source — 13 instances confirmed, the pages-router link chain claimed active was wrong (never executed; recorded as uncertain), explicit Suspense wrappers rejected, and one role of an existing record corrected; three first drafts of Part 3 code were wrong (Java cyclic inheritance, C++ `const` members preventing assignment, error-boundary reset rule) and fixed before the recorded runs. Correct claims include the missing licence, dead caching-proxy code, the client-side PIN check and the 18/19 protected procedures. Reviewer: **[TEAM TO FILL: roll no./name of the team member who re-checked, with date]**

Generated code: All Part 2 production code and tests and all Part 3 code were generated by the model. Use and testing: Part 2 — characterization tests run on the original before refactoring, acceptance tests per change, 65/65 passing plus typecheck on the final snapshot; Part 3 — 64 implementations, 708/708 shared-fixture checks. The team must review and be able to explain this code **[TEAM TO FILL: reviewers per part]**.

6 Findings, contributions and references

Findings:

- netflx-web uses **16 pattern types in 28 collaborations** at the baseline (first pass: 10 / 15). Most come from frameworks and libraries (tRPC, zustand, NextAuth, React Query, Next.js, T3 scaffold) and are configured, not written, by the application; the second pass shows that a first pass limited to the feature areas misses exactly this infrastructure (front controllers, builder, context object, boundaries). Observer (4) is the most frequent; recommendation has no pattern because it has no logic.
- The access requirements fitted the existing Chain of Responsibility idea: one tRPC middleware was added and a new rule chain absorbed premium, region and parental rules with one class each; the chain order (hide before lock) is a design decision of its own and is tested.
- The env Protection Proxy explains a code smell: the plan list was duplicated because the server list reads secrets. Moving the client-safe part into one module removed the duplication and an existing inconsistency.
- Security observations (pre-existing, reported): the profile PIN is checked in the browser; profile and My-List procedures accept any profileId; the server caching proxy is dead code; two tRPC client proxies exist side by side.
- Limitations: no end-to-end run with real Stripe, TMDB and Google services; sample licence and premium data; lizard's TSX parsing is imperfect; Part 3 times are start-up dominated and machine-specific.

Contributions: **[TEAM TO FILL: one line per student: actual contribution and link to the files/commits they produced or reviewed]**

Roll number	Name	Batch	Contribution
AM.SC.U4CSE23161	Swaminathan A	CSE B	**[TEAM TO FILL:]**
AM.SC.U4CSE23207	Anna Ouseph	CSE C	**[TEAM TO FILL:]**
AM.SC.U4CSE23230	Leya Paul	CSE C	**[TEAM TO FILL:]**
AM.SC.U4CSE23242	Pooja Sathyan	CSE C	**[TEAM TO FILL:]**
AM.SC.U4CSE23069	Rahul Rajan	CSE A	**[TEAM TO FILL:]**

References:

[1] E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.

[2] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed., Addison-Wesley, 2018.

[3] sadmann7, netflx-web, https://github.com/sadmann7/netflx-web (commit 676e5dd, accessed 2026-10-10).

[4] Next.js documentation, https://nextjs.org/docs; T3 stack (create-t3-app), https://create.t3.gg.

[5] MDN Web Docs, "Proxy", https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Proxy.

[6] Oracle, Java SE 21 API, `java.lang.reflect.Proxy`, https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/reflect/Proxy.html.

[7] Python 3 Language Reference, Data model — customizing attribute access, https://docs.python.org/3/reference/datamodel.html.

[8] tRPC documentation (procedures and middlewares), https://trpc.io/docs.

[9] zustand (incl. `persist` and `devtools` middleware), https://github.com/pmndrs/zustand.

[10] NextAuth.js documentation (providers, adapters), https://next-auth.js.org.

[11] TanStack Query v4 documentation (caching, invalidation), https://tanstack.com/query/v4.

[12] lizard code complexity analyser, https://github.com/terryyin/lizard; PlantUML, https:

//plantuml.com; vitest, https://vitest.dev.

7 Team folder and datasets (take a screenshots from the GitHub repository— your team)

[TEAM TO FILL: insert a screenshot of the A12 folder in the GitHub repository after upload]

```
A12/
  README.md
  A12_Report.pdf
  A12_Report.tex
  data/
    code/          original application source (676e5dd)
    test_result/   baseline logs, Part 1-3 evidence scripts and logs,
                   raw timings, LLM output records
  Part1/
    patterns.csv
    Pattern_count.csv
    pattern_architecture/   diagram sources (.puml) + images (.png)
    Part1_prompt.txt
    llm.csv
  Part2/
    code/          evolved application source (CH03)
    changes/Change1..3/     requirement, diff, test result (+ patch)
    pattern_evalution.csv
    Pattern_count.csv
    diagrams/      updated UML
    Part2_prompt.text
    llm.csv
  Part3/
    NN_<pattern>_<problem>/   16 folders, each with spec.md, cases.tsv and
      java/ python/ javascript/ cpp/   snippet, test runner, README, log
    llm/
      part3_prompt.txt
      llm.csv
      pattern_crosslanguage.csv
      A12_slide.pdf (+ .tex)
```